LECTURE NOTES · ARCHITECTURE

The Transformer, Visually

Companion notes to the slides

The decoder-only transformer, one component at a time. Tokens, embeddings and position; attention with queries, keys, values, the causal mask and many heads; the MLP, residual connections and normalization; the output head, sampling and the KV cache; training, parameter and compute accounting, and what changed between 2017 and today. One section per slide, with every term defined and the intuition spelled out, and a reading list for each part.

For advanced undergraduates and beginning graduate students

October 2026

These notes follow the deck *The Transformer, Visually* (31 slides) slide by slide. Dimensions and identifiers quoted for named models were checked against the papers and public configurations listed in the reading list; numbers marked “derived” were computed from those configurations; values the slides label “illustrative” (token IDs, attention weights, probabilities) are invented to make a mechanism visible and are reproduced here so the arithmetic can be followed.

Contents

How to use these notes **3**

1 Overview (slides 1–4) **3**
1.1 Slide 1: Title: The Transformer, Visually . . . 3
1.2 Slide 2: An LLM does one thing: predict the next token . . . 4
1.3 Slide 3: The map: a decoder-only transformer . . . 5
1.4 Slide 4: Inside the stream: one lane per token . . . 7

2 Tokens (slide 5) **8**
2.1 Slide 5: Tokenization: text becomes integers . . . 8

3 Embedding (slides 6–7) **10**
3.1 Slide 6: Token embeddings: a lookup table . . . 10
3.2 Slide 7: What the embedding space looks like . . . 11

4 Position (slides 8–9) **12**
4.1 Slide 8: Position: telling the model what came first . . . 12
4.2 Slide 9: Rotary position embedding (RoPE) . . . 13

5 Attention (slides 10–16) **15**
5.1 Slide 10: Attention: each token gathers what it needs from the past . . . 15
5.2 Slide 11: Queries, keys and values . . . 16
5.3 Slide 12: Scaled dot-product attention, in matrices . . . 17
5.4 Slide 13: From scores to weights: a worked example . . . 18
5.5 Slide 14: The causal mask . . . 20
5.6 Slide 15: Multi-head attention . . . 21
5.7 Slide 16: What individual heads learn to do . . . 22

6 MLP (slides 17–18) **24**
6.1 Slide 17: The MLP: where each token is transformed . . . 24
6.2 Slide 18: Activation functions . . . 25

7 Norm and residual (slides 19–20) **27**
7.1 Slide 19: Residual connections and where the norm goes . . . 27
7.2 Slide 20: LayerNorm and RMSNorm . . . 28

8 Block and stack (slides 21–22) **30**
8.1 Slide 21: One complete transformer block . . . 30
8.2 Slide 22: Stacking blocks: depth and width . . . 31

9 Output (slides 23–25) **33**
9.1 Slide 23: The output head: from a vector to a distribution . . . 33
9.2 Slide 24: Choosing the next token: temperature, top-k, top-p . . . 34
9.3 Slide 25: Generation and the KV cache . . . 36

10 Training and scale (slides 26–28) 37
10.1 Slide 26: Training: predict every next token at once 37
10.2 Slide 27: Where the parameters live: GPT-2 small 39
10.3 Slide 28: Compute cost: linear in the weights, quadratic in the context 40

11 The family and the whole machine (slides 29–31) 42
11.1 Slide 29: The family, and what changed since 2017 42
11.2 Slide 30: The whole machine on one slide 43
11.3 Slide 31: Sources and further reading 44

A Notation 46

B Formulas at a glance 47

C Exercises 48

D Reading list by section 50

Index of terms 55

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the equations written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Conventions. T is the number of tokens in the sequence being processed (the context), V the vocabulary size, d the width of the model (the length of the vector that represents one token), L the number of blocks, h the number of attention heads, $d_k = d/h$ the width of one head, and d_{ff} the hidden width of the MLP. Per-token formulas treat a token's vector x_t as a column vector of length d and write a linear map as Wx_t; whole-sequence formulas stack the T token vectors as the rows of a $T \times d$ matrix X, so the same map reads $XW^\top$. The deck writes $Q = XW_Q$ and so on with the transpose absorbed into the symbol; nothing depends on the choice. Positions are indexed $1, \ldots, T$; "earlier" means a smaller index. A *weight* is a learned number; a *parameter* is the same thing counted. Multiply–adds are counted as one operation where the slides say "multiply–adds" and as two floating-point operations (FLOPs) where they say FLOPs.

A reading plan for a single lecture: Section 1 (the map) and Sections 2–4 (tokens, embeddings, position) take the first quarter of the session; Section 5 (attention) is the core and deserves a third; Sections 6–8 (MLP, norm, the block and the stack) another quarter; Sections 9 and 10 (output, generation, training, cost) the rest, with Section 11 as the closing summary. The worked attention example on slides 10–13 and the parameter count on slide 27 are the two places where students should be asked to reproduce the numbers by hand; the exercises in Appendix C do exactly that. Appendix D is the reading list, section by section, and matches the paper archive that accompanies these notes.

1 Overview (slides 1–4)

1.1 Slide 1: Title: The Transformer, Visually

A component-by-component tour of the architecture inside large language models: tokens, embeddings, attention, MLPs, normalization, and the output head.

What the slide shows The cover graphic is the model drawn as a vertical stack, read from the bottom up. Four grey token chips (*The cat sat on*) sit at the base; above them a green plate labelled Embedding and a violet one labelled Position; then three repeated pairs of plates, orange Attention and blue MLP, each pair grouped as one block; then an amber Norm plate and a pink plate labelled Unembed + softmax. An arrow runs up the middle. The same colours label every later slide, and the footer of the deck carries ten stage names: Overview, Tokens, Embedding, Position, Attention, MLP, Norm + residual, Block + stack, Output, Training + scale.

Details The deck is about one architecture, the *decoder-only transformer*, which is the design behind essentially every large language model in use today: the GPT series, Llama, Mistral, Qwen, Gemma, DeepSeek and their relatives. The lecture takes the model apart in the order a token passes through it, and at each stage asks three questions: what is the shape of the data here, what operation is applied, and how many

weights does it take. Three running examples carry the whole deck. The sentence *The cat sat on the* is used wherever a next-token prediction is needed; the sentence *Alice thanked Bob because she* is used wherever attention is explained, because continuing it requires knowing who "she" is; and GPT-2 small, the smallest public model with the modern layout, is used wherever a real number is needed, because its configuration is public and its 124 million parameters can be counted by hand.

The word *transformer* names the architecture introduced by Vaswani et al. in 2017 for machine translation. That model had two halves, an encoder that read the source sentence and a decoder that wrote the translation. Language models keep only the decoder half, and the deck follows that choice: whenever it says "the transformer" it means a stack of decoder blocks with causal attention, which is what slide 29 compares with the other two wirings.

Terms

transformer A neural network built from repeated blocks, each combining an attention sublayer, which moves information between positions in a sequence, with a position-wise MLP sublayer, which transforms each position on its own. Introduced by Vaswani et al. (2017).

large language model (LLM) A transformer trained to predict the next token of text on a very large corpus, typically with billions of parameters and trillions of training tokens.

decoder-only A transformer that uses only the decoder half of the 2017 design: one stack of blocks, with attention restricted so that each position reads only earlier positions. The architecture of GPT, Llama and their relatives.

token The unit of text the model reads and writes: a short string, usually a word or a piece of a word, drawn from a fixed vocabulary and represented inside the model by an integer identifier.

parameter A number stored in the model and adjusted by training; a weight. "124M parameters" means 124 million such numbers.

Intuition

The whole lecture fits in one sentence: a transformer turns a list of token identifiers into a list of vectors, lets the vectors exchange information with each other (attention) and be rewritten on their own (the MLP), repeats that many times, and finally turns the last vector into a probability for every possible next token. Everything else is detail about how each of those steps is done and what it costs. When a later slide seems complicated, place it on that sentence: which step is it, and is it about moving information between positions or about transforming one position?

1.2 Slide 2: An LLM does one thing: predict the next token

Text in, a probability for every token in the vocabulary out. Generation is this step in a loop.

What the slide shows On the left, the five tokens *The cat sat on the* in grey chips, followed by a dashed pink chip holding a question mark, feeding an ink-coloured box labelled "Transformer f_θ". On the right, a bar chart headed $P(\text{next token} \mid \text{context})$: *mat* 0.51, *floor* 0.23, *sofa* 0.13, *roof* 0.07, and a row of dots standing for every other token in the vocabulary. Below the chart, a pink chip *mat* with the label "sample one" and an arrow looping back to the question-mark chip labelled "append, then run again". The probabilities are illustrative.

Details The model computes one function. Its input is a sequence of T token identifiers $x_1, \ldots, x_T$; its output is a vector of V probabilities, one per vocabulary entry, which the deck writes $P(\text{next} \mid x_1, \ldots, x_T)$. V is the vocabulary size, tens of thousands to a few hundred thousand for real models; the six-entry vocabulary

on the slide is a toy so that the bars fit. Everything the model does, from answering questions to writing code, is this function applied repeatedly.

Generation, or *decoding*, is a loop around that function. Run the model on the context, obtain the distribution, choose a token (by taking the most likely one, or by sampling at random with the given probabilities; slide 24 covers the choices), append the chosen token to the context, and run again. The model never outputs a sentence; it outputs one distribution at a time, and the loop outside it produces the text. This matters for two reasons that recur in the deck. First, nothing the model has computed for earlier positions needs to change when a token is appended, which is what makes caching possible (slide 25). Second, training can be done without any human labels, because the next token of any text is its own label (slide 26).

The probability view also fixes what "good" means. A model is better if it assigns higher probability to the tokens that actually occur in text. The number $-\log P(\text{actual next token})$ is the loss at one position; averaged over a corpus it is the quantity training minimizes, and exp of it is the *perplexity*, the effective number of equally likely choices the model is hedging between. A perplexity of 20 means the model is, on average, as uncertain as if it were choosing uniformly among 20 tokens.

output: $P(x_{T+1} = v \mid x_1, \ldots, x_T)$ for every $v \in \{1, \ldots, V\}$, summing to 1
generation: $x_{T+1} \sim P(\cdot \mid x_{\le T})$, then $T \leftarrow T + 1$, repeat
loss at one position: $-\log P(x_{t+1} \mid x_{\le t})$; perplexity $= \exp(\text{mean loss})$

Terms

vocabulary The fixed set of V tokens the model can read or write. Every input is a sequence of vocabulary entries and every output distribution is over the same set.

next-token prediction The task of assigning a probability to each possible token that could follow a given context. The single objective on which language models are trained.

context The tokens the model is given as input at one step: the prompt plus everything generated so far. Its length T is bounded by the model's context window.

sampling Choosing the next token at random according to the model's probabilities (or a modified version of them), as opposed to always taking the most likely token.

autoregressive Producing a sequence one element at a time, with each element conditioned on all the previous ones. The model is autoregressive because its output at each step is fed back as input.

perplexity exp of the average next-token loss; the effective number of equally likely tokens the model is choosing among. Lower is better.

Intuition

Think of the model as a very well-read person asked, over and over, "what single word comes next?" and allowed only to answer with odds. Nothing in the architecture is about sentences, facts or tasks; those appear because predicting the next token well, on enough text, requires knowing them. The question to ask of any claim about what an LLM "does" is: how would that show up in the next-token distribution?

1.3 Slide 3: The map: a decoder-only transformer

One stream of vectors runs left to right; every component reads from it and adds back to it.

What the slide shows The data path of the model as a left-to-right chain of boxes: Tokenizer, then Embed (+ Position), then a bracketed block containing Norm, Attention, a + sign, Norm, MLP and another + sign,

with "$\times L$" beside the bracket, then Norm, Unembed and Softmax. Small labels above the boxes name what each one converts: text to token IDs, ID to vector, the final norm, vector to V logits, logits to probabilities. Below the bracket, two labels name the two workers inside a block: attention "moves information between positions" and the MLP "transforms each position". Three notes at the bottom: the T tokens become a $T \times d$ array that keeps its shape through every block and only the last step widens it to $T \times V$; attention is the only place tokens exchange information; and the same block design is stacked L times, 12 to more than 100, each copy with its own weights.

Details The slide is the table of contents of the deck, and each box is one later section. Reading the shapes is the fastest way to understand the architecture.

- **Tokenizer** (slide 5): text $\to$ a list of T integers in $\{1,\dots,V\}$. No learned weights are used at run time; the tokenizer is a fixed procedure plus a lookup table.
- **Embed + position** (slides 6–9): each integer $\to$ a vector of width d, giving a $T \times d$ matrix. Position information is added here in the original design, or injected inside attention in modern ones.
- **Block** $\times L$ (slides 10–22): a $T \times d$ matrix in, a $T \times d$ matrix out. Each block has two sublayers. The attention sublayer reads across positions: row t of its output depends on rows $1,\dots,t$ of its input. The MLP sublayer reads within a position: row t of its output depends only on row t of its input. Each sublayer is preceded by a normalization and followed by an addition back onto the input (the residual connection). Because the shape never changes, blocks can be stacked without limit, and L is a free design choice.
- **Norm, unembed, softmax** (slide 23): the final $T \times d$ matrix is normalized and multiplied by a $d \times V$ matrix to give $T \times V$ scores, one row per position; the softmax turns each row into probabilities. Only the last row is needed to pick the next token, but during training all T rows are used.

The number L is called the depth. GPT-2 small has $L = 12$; GPT-3 has 96; Llama 3.1 405B has 126 (slide 22). The width d runs from 768 in GPT-2 small to 16,384 in Llama 3.1 405B.

$$X^{(0)} = \mathrm{Embed}(x_{1..T}) \in \mathbb{R}^{T\times d}$$
$$X^{(l)} = \mathrm{Block}_l\left(X^{(l-1)}\right) \in \mathbb{R}^{T\times d} \text{ for } l = 1,\dots,L$$
$$P = \mathrm{softmax}\left(\mathrm{Norm}(X^{(L)})\,W_U\right) \in \mathbb{R}^{T\times V}, \text{ rows summing to 1}$$

Terms

block (layer) One repeated unit of the transformer: an attention sublayer and an MLP sublayer, each with its normalization and residual addition. "Layer" is used for the same thing in most papers; L counts them.

depth The number of blocks L.

width (model dimension) The length d of the vector that represents one token inside the model; also written d_{model}. Every block's input and output is $T \times d$.

embedding The step that turns a token identifier into a vector of width d by looking up a row of a learned table; also the vector itself.

unembedding The step that turns a vector of width d back into one score per vocabulary entry, by multiplying with a $d \times V$ matrix W_U.

softmax The function that turns a vector of real scores into probabilities: $p_i = e^{z_i} / \sum_j e^{z_j}$. Applied to each row of the output, and inside attention to each row of scores.

sublayer One of the two halves of a block: the attention sublayer or the MLP sublayer, each wrapped in a norm and a residual addition.

Intuition

Two things in this picture do all the work, and they split the job cleanly: attention is the only place where position t learns anything about other positions, and the MLP is where most of the weights live and where each position is rewritten using what it has gathered. Everything else, the norms, the residual additions, the embedding and unembedding, is plumbing that keeps the numbers well scaled and the shapes constant. When something is unclear, ask whether it moves information across positions or within one.

1.4 Slide 4: Inside the stream: one lane per token

Attention carries information across lanes, from earlier tokens to later ones. The MLP works within a lane.

What the slide shows Four horizontal lanes, one for each token of *The cat sat on*, running left to right through the depth of the model. Three orange bands labelled Attention cut across all four lanes, one in block 1, one in block 2 and one in block L (with dots for the blocks between); inside each band, small arrows run from every lane to every lane below it, that is, from earlier tokens to later ones, and never upward. Between the bands each lane passes through its own blue MLP box. At the right end each lane ends in a pink chip with the token it predicts: the lane for *The* predicts *cat*, the lane for *cat* predicts *sat*, and so on. Two notes: a lane is one position's vector of width d, and every sublayer reads it and adds its result back, $x \leftarrow x + f(x)$; only attention crosses lanes, and only forward in the sequence.

Details This is the *residual stream* view of the transformer, named in the interpretability literature (Elhage et al. 2021) and the most useful mental model for the rest of the deck. Each position t owns a vector x_t of width d that starts as the token embedding (plus position) and is carried unchanged along a straight path through all L blocks. Every sublayer reads the vector, computes a correction $f(x_t)$ of the same width, and adds it back: $x_t \leftarrow x_t + f(x_t)$. Nothing ever overwrites the stream; it is only ever added to. That additive structure is the residual connection of slide 19, and it has two consequences the deck relies on.

First, the lanes are independent except at attention. The MLP, the norms and the additions each act on one position at a time, with the same weights at every position. The only operation whose output at position t depends on other positions is attention, and the causal mask (slide 14) restricts that dependence to positions $s \le t$. Hence the bands cross lanes forward only: the lane for "on" at position 4 can read "The" at position 1, but nothing computed at position 1 can ever depend on "on". That is also why every lane can carry its own prediction: the lane for "The" predicts "cat" without having seen it, the lane for "cat" predicts "sat", and all T predictions come out of one pass (slide 26). Second, because the final vector at position t is a sum of the embedding plus every correction added along the way, a block late in the stack can read what an early block wrote and build on it. Depth composes.

The slide's $x \leftarrow x + f(x)$ is the equation of the whole stack, written once. Expanding it over the $2L$ sublayers gives $x_t^{(L)} = x_t^{(0)} + \sum_{\text{sublayers}} f(\cdot)$, a sum of $2L$ corrections on top of the embedding. Slide 21 writes out one block's worth, slide 22 stacks them.

Terms

residual stream The vector at one position, carried through the stack and updated by addition at each sublayer. The term comes from the interpretability literature; the deck's lanes are residual streams.

residual connection The addition $x + f(x)$ that routes a sublayer's input past the sublayer and adds the sublayer's output to it, so that the sublayer computes a correction rather than a replacement. Introduced for deep image networks by He et al. (2016).

position-wise An operation applied independently to each position with the same weights: the MLP and the norm are position-wise, attention is not.

causal Respecting the order of time: the computation at position t may depend on positions $1, \ldots, t$ only. Causal attention is attention under that restriction.

Intuition

Picture five people seated in a row, each holding a notebook. In the MLP steps each person edits their own notebook. In the attention steps each person may glance at the notebooks of the people to their left and copy a summary into their own. Nobody's notebook is ever replaced; it only gains pages. After enough rounds, the last person's notebook contains, in some encoded form, everything relevant from the whole row. The test of the picture: if a fact must travel from position 1 to position 5, it needs at least one attention step, and if it must travel from 5 to 1, it cannot travel at all.

2 Tokens (slide 5)

2.1 Slide 5: Tokenization: text becomes integers

The model never sees characters or words, only IDs of subword pieces from a fixed vocabulary.

What the slide shows Three rows. The top row is the string *The transformer is unbelievably fast*. The middle row splits it into eight coloured chips: `The`, `␣transform`, `er`, `␣is`, `␣un`, `believ`, `ably`, `␣fast`, where the open-box symbol marks a leading space that belongs to the token. The bottom row gives each chip an integer: 464, 47385, 263, 318, 555, 6667, 1346, 3049. Both the split and the IDs are illustrative. On the right, a four-step cartoon of byte-pair encoding on the word *lowest*: start from the characters `l o w e s t`, merge the most frequent pair to get `es`, merge again to get `est`, and continue until the vocabulary has V entries, at which point the word is `low` + `est`. Three facts at the bottom: common words are one token and rare words split into pieces so that any string can be encoded; V is about 32k to 256k in current LLMs and 50,257 in GPT-2; and for English one token is about four characters, or three quarters of a word.

Details The tokenizer is the boundary between text and the model, and it is not part of the neural network: it is a deterministic procedure, fixed before training, that maps a string to a list of integers and back. The model only ever sees the integers, so the tokenizer's choices decide what the model can and cannot easily represent. Everything in the deck after this slide is about integers in $\{1, \ldots, V\}$, not characters.

Three levels are possible. Character-level tokens (or raw bytes) give a tiny vocabulary but very long sequences, and attention cost grows with the square of sequence length (slide 28). Word-level tokens give short sequences but an unbounded vocabulary: every misspelling, name and new coinage would be an unknown word. *Subword* tokenization sits between them. Frequent strings, including most common words with their leading space, get a single token; rare strings are broken into pieces that are themselves in the vocabulary; and in the limit every byte value is a token, so no input can fail to encode. The example on the slide shows the pattern: *transformer* becomes `␣transform` + `er`, *unbelievably* becomes three pieces, and the short common words are single tokens.

Byte-pair encoding (BPE) is the way most current vocabularies are built. It was a compression algorithm (Gage 1994) before Sennrich, Haddow and Birch (2016) used it to build translation vocabularies. Training the tokenizer means: start with the alphabet of single characters (or the 256 byte values), count how often each adjacent pair of symbols occurs in a large text sample, merge the most frequent pair into a new symbol, add it to the vocabulary, and repeat until the vocabulary has the desired size. Applying the tokenizer to new text replays the same merges in the same order. GPT-2's vocabulary is 256 byte symbols plus 50,000 learned

merges plus one end-of-text marker, which is where 50,257 comes from; because it starts from bytes rather than characters it is called byte-level BPE, and it can encode any byte string, including text in any script and binary data. Alternatives exist: WordPiece (BERT) chooses merges by a likelihood criterion, and the unigram method (Kudo 2018; the SentencePiece library) starts from a large candidate vocabulary and prunes it. For the rest of the deck the method does not matter; the output is a list of IDs.

Vocabulary sizes have grown. GPT-2 and GPT-3 use 50,257; Llama 2 uses 32,000; Llama 3 uses 128,256; Qwen models use about 152k; Gemma uses 256,000. A larger vocabulary means fewer tokens per text (so shorter sequences and cheaper attention) and better coverage of non-English scripts and code, at the cost of a larger embedding table (slide 6) and a larger output softmax (slide 23). The rule of thumb on the slide is OpenAI's for English: about four characters per token, so 100 tokens is roughly 75 words. Other languages and code are often less efficient, two to four times more tokens for the same content in some scripts, which is one of the reasons vocabularies grew.

Terms

tokenizer The fixed procedure that converts text into a sequence of token identifiers and back. Not learned by gradient descent; built once before training.

subword A token that may be a whole word, a part of a word, or a single character or byte. Subword vocabularies encode any string by falling back to smaller pieces.

byte-pair encoding (BPE) A way to build a subword vocabulary: start from single characters or bytes, repeatedly merge the most frequent adjacent pair into a new symbol, stop at the desired vocabulary size. Encoding replays the merges.

byte-level BPE BPE whose initial alphabet is the 256 byte values, so that any input, in any script, can be encoded. Used by GPT-2 and most models since.

token ID The integer index of a token in the vocabulary, from 1 to V (or 0 to $V-1$). What the model actually receives.

vocabulary size V The number of distinct tokens. 50,257 in GPT-2; 32,000 in Llama 2; 128,256 in Llama 3; 256,000 in Gemma.

leading space In GPT-style tokenizers the space before a word is part of the word's token, so `␣fast` and `fast` are different tokens with different IDs.

end-of-text token A special vocabulary entry marking a document boundary, so that the model learns where texts begin and end. GPT-2's 50,257th entry.

Intuition

A tokenizer is a compression dictionary: frequent strings get one code, rare ones are spelled out in pieces. The model inherits every quirk of that dictionary. Numbers may split into odd chunks (*1234* as *12 + 34*), words with and without a leading space are unrelated IDs, and a word's pieces carry no record of how the pieces relate. When a model does something strange with spelling, counting letters or arithmetic, the first thing to check is how the input was tokenized.

3 Embedding (slides 6–7)

3.1 Slide 6: Token embeddings: a lookup table

Each ID selects one row of a learned matrix E. That row is the token's starting vector.

What the slide shows On the left, three tokens *The*, *cat*, *sat* with illustrative IDs 464, 3797, 7731. In the middle, a tall green matrix E with V rows and d columns, "one row per vocabulary entry"; each ID points to one highlighted row. On the right, the three selected rows laid out as the rows of a smaller matrix $X \in \mathbb{R}^{T\times d}$, "one row per token", labelled x_1, x_2, x_3. Two notes: $x_t = E[\mathrm{id}_t]$ is indexing, not arithmetic (equivalent to a one-hot vector times E); and the table is learned with everything else, which in GPT-2 small means $50{,}257 \times 768 = 38.6$M parameters, 31% of the model.

Details The embedding table E is a matrix with one row per vocabulary entry and d columns. Looking up token t means copying row id_t into position t of the input matrix X. Writing the token as a one-hot vector e_{id_t} (a length-V vector with a single 1) makes the lookup a matrix product, $x_t = e_{\mathrm{id}_t}^\top E$, which is the form in which the gradient is derived: during training, the gradient of the loss with respect to x_t flows back into exactly one row of E, so each row is adjusted only when its token appears. Rows of rare tokens are updated rarely and are typically worse trained.

The width d is the single most important architectural number. It is the length of every token's vector at every layer, and the block weights scale as d^2 (slide 21). GPT-2 small has $d = 768$; the 2017 base model had 512; Llama 3 8B has 4,096, Llama 3 70B 8,192, Llama 3.1 405B 16,384.

The parameter count is Vd. For GPT-2 small, $50{,}257 \times 768 = 38{,}597{,}376$, which is 31.0% of its 124,439,808 parameters (slide 27). The share falls quickly with model size because the table grows with d while the blocks grow with Ld^2: Llama 3 8B's table is $128{,}256 \times 4{,}096 = 525$M, about 6.5% of 8B, and its 405B sibling's table is 2.1B, about half a percent. The embedding is also where weight tying enters (slide 23): GPT-2 reuses E as the output matrix, so the table is counted once.

At this stage the vector for a token is the same wherever the token occurs. Position has not been added, and nothing from the context has been read. The next slide says what the rows look like after training; slide 8 adds position; the blocks do the rest.

$$E \in \mathbb{R}^{V\times d}, \quad x_t = E[\mathrm{id}_t] = e_{\mathrm{id}_t}^\top E, \quad X = [x_1; \ldots; x_T] \in \mathbb{R}^{T\times d}$$

parameters: Vd (GPT-2 small: $50{,}257 \times 768 = 38{,}597{,}376$)

Terms

embedding table The learned matrix $E \in \mathbb{R}^{V\times d}$ whose row i is the input vector for token i.

one-hot vector A vector of length V with a single 1 at the token's index and zeros elsewhere. Multiplying it by E selects a row, which is why embedding lookup is a linear layer in disguise.

lookup Reading a row of a table by index; the embedding is a lookup, with no multiplication.

input matrix X The $T \times d$ matrix whose rows are the embedded tokens of the context; the first state of the residual stream.

Intuition

The embedding table is the model's dictionary, except that each entry is not a definition but a point in a d-dimensional space, and the "meaning" of the point is nothing but where it sits relative to the other points. Training moves the points. The question to ask when a model mishandles a rare word is how often that word's row was ever updated.

3.2 Slide 7: What the embedding space looks like

Training arranges tokens so that geometry reflects usage: neighbors are similar, directions are relations.

What the slide shows A two-dimensional cartoon of the d-dimensional embedding space. Three shaded clusters: animals (*cat*, *dog*, *horse*, *lion*) in green, cities (*Paris*, *Tokyo*, *Rome*, *Cairo*) in blue, number words (*two*, *three*, *seven*, *ten*) in amber. Below them a parallelogram: arrows from *man* to *woman* and from *king* to *queen*, drawn parallel and labelled "same offset". Four notes: neighbours are tokens used in similar contexts; consistent offsets can encode a relation, $king - man + woman \approx queen$ (word2vec, Mikolov et al. 2013); the vectors are not yet contextual, so "bank" gets the same input vector by a river or in finance and the layers above resolve it; and d is 768 in GPT-2 small and 4,096 to 16,384 in current large models.

Details Nothing in the training objective says what the embedding space should look like; the structure on the slide is what gradient descent produces when the only goal is next-token prediction. The mechanism is simple. Two tokens that tend to appear in the same contexts (*cat* and *dog* after *the*, before *sat*) receive similar gradients and drift toward similar vectors, because a vector that works for one works for the other. The result is that distance in the space tracks interchangeability of usage, which correlates with meaning.

The parallelogram is the famous observation from word2vec (Mikolov et al. 2013), a model that learned word vectors from context prediction before transformers existed: the vector difference between *man* and *woman* is approximately the same as between *king* and *queen*, so that $king - man + woman$ lands near *queen*. The relation "male to female" is a direction. Many relations behave this way to some degree (country to capital, singular to plural, comparative forms), and the same regularities appear in transformer embedding tables, with caveats: the analogies hold approximately, for frequent words, and the nearest neighbour of the computed point is often one of the inputs themselves unless they are excluded.

The important limitation is the third note. The embedding table assigns one vector per token, so a token with several senses gets one vector that has to serve all of them. Resolving which sense is meant requires reading the context, and the first place that can happen is the first attention layer. The vectors at the output of later blocks are *contextual*: the vector at the position of *bank* after several blocks differs depending on whether *river* or *loan* appeared earlier. The static table is only the starting point.

The cartoon is two-dimensional because paper is. Real tables have hundreds to thousands of dimensions, and two-dimensional pictures of them are projections (by principal components or similar) that keep a small fraction of the structure. Clusters that look overlapping in a projection may be far apart in the full space.

Terms

embedding space The d-dimensional space in which token vectors live; distances and directions in it are learned, not designed.

word2vec A 2013 family of shallow models (Mikolov et al.) that learn word vectors by predicting context words; the origin of the analogy-by-vector-arithmetic observation.

analogy (vector arithmetic) The observation that a relation between word pairs appears as a consistent vector offset, so $king - man + woman \approx queen$.

contextual representation A vector for a token that depends on the surrounding tokens, as produced by the blocks; the embedding table's vectors are not contextual.

polysemy A token having several meanings. The embedding table cannot separate them; the layers above can.

Intuition

The embedding table knows what a word is like on average; the rest of the network figures out what it means here. A useful mental test for whether a property can live in the table: would it be true of the token in every sentence it ever appears in? "*cat* is an animal" passes. "*bank* is a financial institution" fails, and must be decided upstairs.

4 Position (slides 8–9)

4.1 Slide 8: Position: telling the model what came first

Attention treats its input as a set. Order has to be injected explicitly.

What the slide shows On the left, the sentences *dog bites man* and *man bites dog*, both feeding the same dashed bag containing the three chips in no order, with the caption that without position information both sentences are the same bag of vectors. On the right, a heat map of the sinusoidal position encoding for $d = 16$ shown: positions $t = 0, 1, 2, \ldots$ run left to right, the 16 dimensions run top to bottom, and the colour is the value of $PE_{t,j}$. The top rows oscillate fast, the bottom rows slowly. Under it the formula $PE_{t,2i} = \sin(t\omega_i)$, $PE_{t,2i+1} = \cos(t\omega_i)$, $\omega_i = 10000^{-2i/d}$. Along the bottom, the three schemes: fixed sinusoids added to the embedding (original Transformer, 2017); a learned table $P[t]$ added the same way (BERT, GPT-2); rotary (RoPE), applied inside attention (next slide).

Details The problem is a consequence of how attention works (slides 11–12). Attention computes, for each query position, a weighted average over the other positions, with weights that depend only on the *content* of the vectors, not on where they are. If the input vectors were permuted, the attention output would be permuted the same way and otherwise unchanged; the MLP and the norms are position-wise, so they have the same property. A transformer with no position information is therefore a function of the *set* of input vectors: *dog bites man* and *man bites dog* would produce the same next-token distribution. (The causal mask breaks the symmetry slightly, because position t sees t vectors and position $t+1$ sees one more, and models trained without any explicit position encoding do learn some order information from that; but every production model injects position explicitly.)

The first fix, from the 2017 paper, is to add to each token's embedding a vector that depends only on its position. The sinusoidal encoding builds that vector from $d/2$ sine–cosine pairs at geometrically spaced frequencies. Pair i oscillates with angular frequency $\omega_i = 10000^{-2i/d}$: pair 0 completes a cycle every 2π positions, and the last pair has a period of about $2\pi \cdot 10000$ positions. Together the pairs form a code that is different at every position within any realistic range and, more usefully, has a fixed-offset property: PE_{t+k} is a linear function of PE_t for every k, because shifting the position by k rotates every pair by a fixed angle $k\omega_i$. The authors conjectured this would let the model attend by relative position, and it extrapolates to positions longer than any seen in training, at least in principle. The heat map on the slide is this code with $d = 16$: fast stripes at the top, slow gradients at the bottom.

The second fix, used by BERT and GPT-2, replaces the formula by a learned table $P \in \mathbb{R}^{T_{\max} \times d}$ with one row per position up to the maximum context $T_{\max}$; the model adds $P[t]$ to x_t and learns the rows like any other weights. GPT-2 small's table is $1{,}024 \times 768 = 786{,}432$ parameters, 0.6% of the model (slide 27). Learned tables are simple and work well within the trained length, but they have no row for position $T_{\max} + 1$, so the context cannot be extended without retraining.

The third fix, rotary position embedding, is the one current models use, and it is different in kind: nothing is added to the embedding. Instead, position is injected inside every attention layer, by rotating the query and key vectors by an angle proportional to their positions. The next slide explains why that makes attention scores depend only on relative position.

sinusoidal: $PE_{t,2i} = \sin(t\,\omega_i)$, $PE_{t,2i+1} = \cos(t\,\omega_i)$, $\omega_i = 10000^{-2i/d}$, $i = 0, \ldots, d/2-1$
input with position (additive schemes): $x_t = E[\mathrm{id}_t] + PE_t$ or $x_t = E[\mathrm{id}_t] + P[t]$

Terms

permutation invariance The property that reordering the inputs reorders the outputs identically; attention without position information has it, which is why order must be injected.

position encoding Any scheme that makes the model's computation depend on where a token is in the sequence: a vector added at the input, or a rotation applied inside attention.

sinusoidal encoding The fixed position vectors of the 2017 Transformer, built from sines and cosines at $d/2$ geometrically spaced frequencies.

learned position table A matrix $P \in \mathbb{R}^{T_{\max} \times d}$ of trainable position vectors, one per position up to the maximum context length; used by BERT and GPT-2.

maximum context length $T_{\max}$ The longest sequence the model is built or trained to handle: 1,024 tokens for GPT-2, 2,048 for GPT-3, 4,096 for Llama 2, 8,192 for Llama 3 (extended to 128k in Llama 3.1).

absolute position Position counted from the start of the sequence; additive schemes encode it directly.

relative position The distance $m - n$ between two positions; what attention actually needs for most linguistic purposes, and what RoPE encodes directly.

Intuition

Attention is a lookup by content, like searching a pile of index cards by what is written on them; the cards have no inherent order. Position encoding writes a page number on every card, in a form the model can read. The sinusoidal scheme writes the page number as the hands of several clocks running at different speeds: the fast hands tell neighbouring positions apart, the slow hands tell distant ones apart, and the difference between two page numbers can be read off from the angle between the hands. RoPE keeps the clocks but turns the cards themselves instead of writing on them.

4.2 Slide 9: Rotary position embedding (RoPE)

Encode position as a rotation. Attention scores then depend only on how far apart two tokens are.

What the slide shows Two rows of four dials. Each dial is a circle with a hand; the row labelled "a fast pair: large θ_i" turns the hand by a large angle from one position to the next ($m = 0, 1, 2, 3$), the row labelled "a slow pair: small θ_i" by a small one. A caption: each pair of dimensions is a clock hand; at position m it has turned by $m\theta_i$, with $\theta_i = 10000^{-2i/d}$; nothing is learned. On the right, the two equations $\tilde{q}_m = R(m\theta)\,q$, $\tilde{k}_n = R(n\theta)\,k$ and the property $\langle \tilde{q}_m, \tilde{k}_n \rangle = \langle R((m-n)\theta)\,q, k \rangle$. Three notes: the scheme is relative by construction, since shifting both tokens by the same amount leaves the score unchanged; it acts on q and k in every attention layer, leaving values and the residual stream untouched; and it is used by the Llama, Mistral, Qwen and Gemma families (Su et al. 2021).

Details RoPE (Su et al. 2021) takes the sinusoidal idea, geometrically spaced frequencies, and moves it from the input to the attention scores. Recall (slides 11–12) that attention compares a query q_m at position m with a key k_n at position n through their dot product. RoPE pairs up the coordinates of these vectors, $(q^{(1)}, q^{(2)}), (q^{(3)}, q^{(4)}), \ldots$, treats each pair as a point in the plane, and rotates pair i by the angle $m\theta_i$ before the dot product is taken, with $\theta_i = 10000^{-2i/d}$ exactly as in the sinusoidal scheme. (Here d is the width of the

vector being rotated, which in multi-head attention is the head width d_k; the base 10,000 is a design constant, and Llama 3 raised it to 500,000 to support longer contexts.) Writing $R(\alpha)$ for the block-diagonal matrix that rotates pair i by $\alpha\theta_i$, the rotated query and key are $\tilde{q}_m = R(m)\,q_m$ and $\tilde{k}_n = R(n)\,k_n$.

The property that makes this work is a fact about rotations: $R(m)^\top R(n) = R(n-m)$, because rotating back by m and forward by n is a rotation by $n-m$. Hence

$$\langle \tilde{q}_m, \tilde{k}_n \rangle = q_m^\top R(m)^\top R(n)\,k_n = q_m^\top R(n-m)\,k_n = \big\langle R(m-n)\,q_m,\ k_n \big\rangle,$$

which depends on the positions only through their difference $m-n$. Shift the whole sequence by 100 positions and every attention score is unchanged. Absolute position is still encoded in each rotated vector, but the only thing attention can measure is relative distance, which is what language mostly needs: "the previous token", "three tokens back", "anywhere earlier". As in the sinusoidal scheme, fast pairs (small i, large θ_i) resolve short distances and slow pairs resolve long ones, and the dot product of a fast pair decays and oscillates with distance while slow pairs stay coherent over long ranges.

Three practical points. RoPE touches only q and k: the values v, the residual stream and the MLP never see position, so position information does not accumulate in the stream and does not have to be learned away. It has no parameters, so it costs nothing in weights and little in compute (a few multiplies per coordinate). And because the angles grow without bound, the model has seen only angles up to $T_{\max}\theta_i$ during training; extending the context beyond that works poorly until the angles are rescaled, which is what position interpolation (Chen et al. 2023), NTK-aware scaling and YaRN do, and why a larger base frequency helps.

pair i of q at position m: $\begin{pmatrix} \tilde{q}^{(2i)} \\ \tilde{q}^{(2i+1)} \end{pmatrix} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix} \begin{pmatrix} q^{(2i)} \\ q^{(2i+1)} \end{pmatrix}, \quad \theta_i = 10000^{-2i/d}$

score: $\langle \tilde{q}_m, \tilde{k}_n \rangle = \langle R(m-n)\,q_m, k_n \rangle$, a function of $m-n$ only

Terms

rotary position embedding (RoPE) Encoding position by rotating each pair of coordinates of the query and key vectors by an angle proportional to the position, so that attention scores depend only on relative position (Su et al. 2021).

rotation matrix The 2×2 matrix $\left(\begin{smallmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{smallmatrix}\right)$ that turns a point in the plane by angle α; $R(\alpha)^\top = R(-\alpha)$ and $R(\alpha)R(\beta) = R(\alpha+\beta)$.

base frequency The constant 10,000 in $\theta_i = 10000^{-2i/d}$, which sets how slowly the slowest pair turns; larger bases (Llama 3: 500,000) stretch the usable context.

query and key The two vectors attention compares to decide how much one position reads from another; defined on slide 11. RoPE acts on exactly these.

context extension Making a trained model usable at longer contexts than it was trained on; for RoPE models, done by rescaling the rotation angles (position interpolation, YaRN) and a little further training.

Intuition

Two people are each wearing a watch that was set to zero at the start of the text and runs at the same speed; each one's watch shows their own position. RoPE's trick is that attention never reads either watch, only the angle between the two hands, and that angle is the same whether the pair sits at positions 3 and 7 or 1003 and 1007. Several watches at different speeds give both fine and coarse readings of the distance. The failure mode follows from the picture: a watch that has never turned past a certain angle during training produces unfamiliar readings beyond it, which is the long-context problem.

5 Attention (slides 10–16)

5.1 Slide 10: Attention: each token gathers what it needs from the past

A weighted average over earlier positions, with weights computed from the content of the tokens.

What the slide shows Two rows of the sentence *Alice thanked Bob because*, each ending in a highlighted fifth token: *she* in the top row, *he* in the bottom row. From each of the four earlier chips an arc runs to the fifth, and the width of the arc is the attention weight written beside it. For *she* the weights are .62 on *Alice*, .07 on *thanked*, .11 on *Bob*, .08 on *because*; for *he* they are .12, .08, .60, .08, with the heavy arc now on *Bob*. Three notes: to continue after "she" the model has to know who "she" is; the position holding "she" pulls in information from earlier positions, and arrow width is the weight; and content decides, because changing one word reroutes the weights with the same parameters. The weights are illustrative. (The slide draws four arcs; the fifth position also attends to itself, with the remaining weight .12, as slides 11 and 13 show.)

Details This slide states what attention does before the next two say how. At each position, attention produces a new vector that is a weighted average of vectors from positions up to and including that one, with weights that sum to one. The weights are not fixed by position or by any rule: they are computed from the vectors themselves, so the same weights can route to *Alice* in one sentence and to *Bob* in another. That is the entire difference between attention and, say, a fixed window average or a convolution: the pattern of who reads from whom is data-dependent.

The example is chosen so that the right answer is unambiguous and requires reading back. To predict what follows *Alice thanked Bob because she*, the model needs to know that "she" refers to Alice (so a continuation like *was grateful* is likely and *had helped her* is not), and the only way for position 5 to obtain that is to read position 1. The weights on the slide show a position that has done so: 62% of its average comes from *Alice*. Replace *she* with *he* and the reference flips to Bob; the model's parameters are identical in the two rows, only the input differs, and so the weights differ. In a trained model this routing is done by heads specialised for coreference (slide 16); here it is drawn by hand.

Two properties already visible here recur throughout. The weights are a probability distribution over the readable positions, which is what makes the output a weighted *average* rather than a sum that grows with context length. And the readable positions are only the earlier ones (and the current one): the arcs point forward in the sentence, never backward, which is the causal mask of slide 14.

Terms

attention An operation that, for each position, forms a weighted average of vectors from the readable positions, with weights computed from the vectors' content. The only operation in the block that mixes positions.

attention weight The coefficient α_{ij} with which position i's output includes position j's value; non-negative, and summing to one over j for each i.

coreference The relation between a pronoun or phrase and the earlier mention it refers to ("she" and "Alice"). Resolving it is a job attention can do.

data-dependent routing Deciding where information flows from based on the input itself rather than on fixed wiring; the defining property of attention.

self-attention Attention in which the queries, keys and values all come from the same sequence (the decoder's own stream), as opposed to cross-attention from a separate encoder (slide 29).

Intuition

Attention is a question asked by one position and answered by all the earlier ones at once, with the answers blended in proportion to how well each position matches the question. "She" asks, roughly, "who is the female person mentioned before me?" and *Alice* matches best. When a model gets a reference wrong, the picture says where to look: either no position asked the right question, or the answer was blended with too much else.

5.2 Slide 11: Queries, keys and values

A soft dictionary lookup: compare one query with every key, then blend the values in proportion to the match.

What the slide shows A left-to-right pipeline for the single query position *she*. On the left, the chip *she* is mapped by W_Q to a four-cell query vector q_i in orange, captioned "what am I looking for?". Five rows follow, one per token *Alice*, *thanked*, *Bob*, *because*, *she*: each has a violet key vector k_j, a score $q_i \cdot k_j$ (3.1, 0.9, 1.4, 1.0, 1.5), a weight α_{ij} after the softmax (.62, .07, .11, .08, .12) drawn as an orange bar, and a teal value vector v_j. Lines of thickness proportional to the weight carry the values into a Σ node, and the sum is a single teal vector "added to the stream at 'she' ". Along the bottom the three roles: query $q_i = W_Q x_i$, what am I looking for; key $k_j = W_K x_j$, what do I contain; value $v_j = W_V x_j$, what do I hand over if chosen.

Details Attention computes its weights by comparing vectors, and it needs three different vectors per position because the three roles are different. From the stream vector x_j at position j, three learned linear maps produce

$$q_j = W_Q x_j, \qquad k_j = W_K x_j, \qquad v_j = W_V x_j,$$

each of width d_k (equal to d for a single head, d/h for one of h heads; slide 15). The *query* is what a position uses to search; the *key* is what a position advertises to be found by; the *value* is what a position hands over when it is chosen. A query and a key are compared by their dot product: $s_{ij} = q_i \cdot k_j$ is large when the two vectors point the same way, and since W_Q and W_K are learned, "the same way" means whatever the training found useful, for example "a query looking for a female name" and "a key saying I am a female name".

The scores for one query are turned into weights by the softmax: $\alpha_{ij} = e^{s_{ij}} / \sum_{j'} e^{s_{ij'}}$. The slide's numbers are a worked case. With scores $(3.1, 0.9, 1.4, 1.0, 1.5)$ the exponentials are $(22.2, 2.46, 4.06, 2.72, 4.48)$, their sum is 35.9, and dividing gives $(0.62, 0.07, 0.11, 0.08, 0.12)$. The softmax does two things at once: it makes every weight positive and makes them sum to one, and it exaggerates differences, so that a score lead of 1.6 (3.1 over 1.5) becomes a weight ratio of five. The output at position i is then the weighted sum of values,

$$z_i = \sum_j \alpha_{ij} v_j,$$

a vector of width d_k that is projected back to width d (by W_O, slide 15) and added to the residual stream at position i.

The dictionary analogy in the subtitle is exact up to softness. A hash table takes a query, finds the one key that matches exactly, and returns its value. Attention takes a query, scores every key, and returns a blend of all values weighted by match quality; when one key dominates, the blend is nearly that one value, and when several match, the result is their average. The split into keys and values is what lets a position be found by one kind of information (its key) and contribute another (its value): *Alice* can be found because it is a name, and contribute, say, "female, subject of the sentence".

$$q_i = W_Q x_i,\ k_j = W_K x_j,\ v_j = W_V x_j, \quad W_Q, W_K, W_V \in \mathbb{R}^{d_k \times d}$$

$$s_{ij} = q_i \cdot k_j, \quad \alpha_{ij} = \frac{e^{s_{ij}}}{\sum_{j' \le i} e^{s_{ij'}}}, \quad z_i = \sum_{j \le i} \alpha_{ij} v_j$$

worked: scores $(3.1, 0.9, 1.4, 1.0, 1.5) \to$ weights $(0.62, 0.07, 0.11, 0.08, 0.12)$

Terms

query The vector $q_i = W_Q x_i$ a position uses to search the others; "what am I looking for?"

key The vector $k_j = W_K x_j$ a position is matched by; "what do I contain?"

value The vector $v_j = W_V x_j$ a position contributes to the output of whoever reads it; "what do I hand over if chosen?"

dot product $q \cdot k = \sum_c q_c k_c$; the similarity measure attention uses, large when the vectors are aligned and long.

score The raw similarity $s_{ij} = q_i \cdot k_j$ (scaled, slide 12) before the softmax; also called a logit of attention.

projection matrices W_Q, W_K, W_V (and W_O): the learned linear maps that produce queries, keys and values from the stream and map the result back.

soft lookup Retrieval that returns a weighted blend of all entries rather than a single exact match; attention is a soft dictionary.

Intuition

Imagine every position holds up a sign (its key) and keeps a sealed envelope (its value). A position that wants information reads all the signs with its own question in mind (its query), decides how much to trust each sign, and opens the envelopes in those proportions. Queries, keys and values are three different projections of the same vector because what makes a position easy to find is not the same as what it is useful to pass on.

5.3 Slide 12: Scaled dot-product attention, in matrices

All T queries are handled at once: three projections, one matrix product, a mask, a softmax, one more product.

What the slide shows At the top, the equation $\text{Attention}(Q,K,V) = \text{softmax}\left(QK^\top/\sqrt{d_k} + M\right)V$ with Q orange, K violet, V teal. Below it, a left-to-right chain of small matrices: the (T,d) input X after Norm; three projections W_Q, W_K, W_V producing Q, K, V, each (T,d_k); the (T,T) score matrix $QK^\top/\sqrt{d_k}$ with the deck's running numbers; the same matrix after $+M$ with the upper triangle marked "future hidden"; the matrix A after the softmax, "rows sum to 1"; a $\times$ node where V enters from below; and the (T,d_k) output $Z = AV$, "to W_O, then added to the stream". Two notes: why $\sqrt{d_k}$ (unscaled dot products grow with d_k and saturate the softmax, shrinking its gradients, which is the paper's stated motivation), and why it is fast (everything is dense matrix algebra, so all positions are processed in parallel on a GPU).

Details The previous slide handled one query. Stacking the T queries as rows of $Q \in \mathbb{R}^{T \times d_k}$, and likewise K and V, turns the per-position formulas into four matrix operations. $QK^\top$ is a $T \times T$ matrix whose entry (i,j) is the score $q_i \cdot k_j$; row i holds all the scores for query i. Dividing by $\sqrt{d_k}$ rescales them. Adding the mask M, which is 0 where $j \le i$ and $-\infty$ where $j > i$, removes the future (slide 14). The softmax applied to each row gives the weight matrix A, with non-negative entries and rows summing to one. Finally $Z = AV$ is $T \times d_k$, and row i of it is $\sum_j \alpha_{ij} v_j$, exactly the previous slide's output. The whole computation for all positions

is two matrix products and an elementwise softmax, which is why attention maps so well onto GPUs: the T queries are not processed in a loop but as one batch.

The scaling by $\sqrt{d_k}$ is a numerical precaution. If the coordinates of q and k are independent with zero mean and unit variance, their dot product has variance d_k, so its typical size grows like $\sqrt{d_k}$. With $d_k = 64$ the raw scores would be eight times larger than needed, pushing the softmax into its saturated regime where one weight is nearly 1 and the rest nearly 0 and the gradients through it are tiny. Dividing by $\sqrt{d_k}$ restores unit variance regardless of head width. The 2017 paper gives exactly this reason: at large d_k the dot products grow large in magnitude and push the softmax into the region where its gradients are extremely small.

The shapes on the slide carry the cost accounting used later. Producing Q, K, V costs three products of a (T,d) by a (d,d_k) matrix: $3Tdd_k$ multiply–adds, which is $3Td^2$ for a full-width head or, summed over h heads of width d/h, still $3Td^2$. The scores cost T^2d_k and the mixing AV another T^2d_k: over all heads, $2T^2d$. That is the $2T^2d$ term of slide 28, quadratic in the context length, against the $12Td^2$ of the weight products, linear in it. The $T \times T$ matrix is also the memory cost: at $T = 8{,}192$ and $h = 32$ heads it has two billion entries per layer, which is what FlashAttention (slide 28) avoids materialising.

$$Q = XW_Q^\top,\ K = XW_K^\top,\ V = XW_V^\top \in \mathbb{R}^{T\times d_k}$$
$$S = QK^\top/\sqrt{d_k} + M \in \mathbb{R}^{T\times T}, \quad A = \mathrm{softmax}_{\mathrm{rows}}(S), \quad Z = AV \in \mathbb{R}^{T\times d_k}$$

cost per layer (all heads): projections $3Td^2$, scores and mixing $2T^2d$, output projection Td^2 multiply–adds

Terms

scaled dot-product attention The 2017 formula $\mathrm{softmax}(QK^\top/\sqrt{d_k}+M)V$: dot-product scores, scaled by $\sqrt{d_k}$, masked, softmaxed by row, applied to the values.

score matrix The $T \times T$ matrix $QK^\top/\sqrt{d_k}$ of all query–key similarities; row i belongs to query i.

attention matrix A The score matrix after masking and row-wise softmax; entry (i,j) is the weight with which position i reads position j.

mask M A matrix added to the scores before the softmax, with $-\infty$ in forbidden entries so that their weights become exactly zero; the causal mask forbids $j > i$.

softmax saturation The regime in which one score is so much larger than the rest that its weight is nearly 1 and the softmax's gradient is nearly zero; the reason for the $\sqrt{d_k}$ scaling.

head width d_k The width of the query, key and value vectors; d for a single head, d/h for each of h heads.

FlashAttention An implementation of exact attention that never writes the full $T \times T$ matrix to memory, computing the softmax in tiles instead (Dao et al. 2022).

Intuition

The matrix form is the per-position form with the loop over queries written as a batch, and that is the whole reason transformers won: the expensive part of the model is a few large dense matrix products, which is the one thing accelerators do at full speed. The $\sqrt{d_k}$ is a reminder that the softmax is only useful in its middle range, where it can express "mostly this, some of that"; scores that are too large turn it into a hard pick and freeze learning.

5.4 Slide 13: From scores to weights: a worked example

Row i of the matrix is token i's query. After masking and softmax it is a probability distribution over what it may read.

What the slide shows Three 5×5 panels for the sentence *Alice thanked Bob because she*, rows labelled by the reader (query) and columns by the position read. Panel 1, "scores $QK^\top/\sqrt{d_k}$", holds the raw numbers, for example row *she* $= (3.1, 0.9, 1.4, 1.0, 1.5)$ and row *thanked* $= (1.8, 1.2, 0.9, 1.6, 0.5)$. Panel 2, "mask the future", keeps the lower triangle and writes $-\infty$ above the diagonal. Panel 3, "softmax each row", shows the weights: row *Alice* $= (1.00)$, row *thanked* $= (.65, .35)$, row *Bob* $= (.14, .35, .51)$, row *because* $= (.18, .39, .19, .24)$, row *she* $= (.62, .07, .11, .08, .12)$, with zeros above the diagonal; each row sums to one, and the last row is boxed. Two notes: adding $-\infty$ above the diagonal makes those weights exactly zero after the softmax; and the last row puts 0.62 of its attention on *Alice*, the weights of the previous slides.

Details The slide is the arithmetic of slide 12 done by hand on the running example, and it is worth reproducing in full because every number can be checked. The scores are illustrative values standing in for $q_i \cdot k_j/\sqrt{d_k}$. The mask keeps entry (i, j) when $j \le i$ and replaces it by $-\infty$ otherwise, so each row i keeps its first i entries. The softmax of a row with $-\infty$ entries gives those entries $e^{-\infty} = 0$ exactly, and the remaining entries are normalised among themselves.

Row by row:

- *Alice* (row 1) can read only itself. One entry, weight 1.00.
- *thanked* (row 2): scores $(1.8, 1.2)$, exponentials $(6.05, 3.32)$, sum 9.37, weights $(0.65, 0.35)$.
- *Bob* (row 3): scores $(0.6, 1.5, 1.9)$, exponentials $(1.82, 4.48, 6.69)$, sum 12.99, weights $(0.14, 0.35, 0.51)$.
- *because* (row 4): scores $(0.8, 1.6, 0.9, 1.1)$, exponentials $(2.23, 4.95, 2.46, 3.00)$, sum 12.64, weights $(0.18, 0.39, 0.19, 0.24)$.
- *she* (row 5): scores $(3.1, 0.9, 1.4, 1.0, 1.5)$, exponentials $(22.2, 2.46, 4.06, 2.72, 4.48)$, sum 35.9, weights $(0.62, 0.07, 0.11, 0.08, 0.12)$.

Three things to notice. First, the diagonal is always readable: a position attends to itself, and in row 1 that is all there is, so the first token's attention output is just its own value, whatever its query says. Second, the softmax is shift-invariant: adding the same constant to every score in a row leaves the weights unchanged, which is why only differences between scores matter, and why a 1.6 lead in row 5 is decisive while a 0.6 lead in row 2 is not. Third, the weights are a distribution, so a row that spreads its attention (row 4, with no weight above 0.39) produces an average of several values, and a row that concentrates it (row 5) produces nearly one value. Both are useful: "look at the previous few tokens" and "find the antecedent" are different jobs (slide 16).

In a real model each row would be the output of the trained W_Q and W_K, and would differ from head to head and layer to layer; what stays fixed is the mechanics shown here.

Terms

row-wise softmax Applying the softmax separately to each row of the score matrix, so that each query's weights sum to one independently of the other queries.

lower triangle The entries (i, j) with $j \le i$ of the attention matrix; the only ones a causal model may use.

shift invariance (of softmax) $\text{softmax}(s + c) = \text{softmax}(s)$ for any constant c: only differences between scores affect the weights.

reader (query position) The row index i: the position whose output is being formed. The deck labels

rows “reader” and columns “position read”.

Intuition

Read the attention matrix one row at a time, and each row answers one question: “where is this token looking?” Reading a column instead answers a different one: “who is looking at this token?” A column with many heavy entries is a position everyone reads, which in trained models is often the first token (slide 16). The exercise that makes the mechanics stick is to change one score in panel 1 and recompute the row by hand.

5.5 Slide 14: The causal mask

Position t may read positions $1,\dots,t$ and nothing later. This single constraint makes the model autoregressive.

What the slide shows On the left, the five chips of the running sentence with *Bob* (position 3) highlighted as the query. Solid orange arcs come in from *Alice* and *thanked*, labelled “allowed: earlier tokens”; dashed grey arcs from *because* and *she* are crossed out, “blocked: these tokens come later”. On the right, the 5×5 mask as a grid with query position t down the side and key position s along the top: zeros on and below the diagonal, $-\infty$ above it, row 3 boxed, and the rule $M_{ts} = 0$ if $s \le t$, otherwise $-\infty$. Two consequences at the bottom. Training: one forward pass yields T next-token predictions, and none of them can see its own answer. Inference: earlier positions never depend on later ones, so their keys and values are computed once and cached.

Details The causal mask is one line of code with consequences for everything else in the deck. Formally it is the matrix M added to the scores on slide 12, $M_{ts} = 0$ for $s \le t$ and $-\infty$ for $s > t$; after the softmax, every forbidden weight is exactly zero, so position t's output is a function of positions $1,\dots,t$ only. Since the MLP and norms are position-wise, the whole model inherits the property: nothing computed at position t, in any layer, depends on any later token.

The first consequence is what makes training efficient. The model's job at position t is to predict token $t+1$. If position t could read position $t+1$ it would simply copy the answer, and the training signal would be worthless. With the mask, a single forward pass over a sequence of T tokens produces T honest predictions, one per position, each made without access to its own target, and all T losses are computed and backpropagated together (slide 26). Without the mask one would have to run the model T times on growing prefixes to get the same T predictions.

The second consequence is what makes generation efficient. When a token is appended to the context, nothing computed for the earlier positions changes, because none of it depended on the new token. Their keys and values at every layer can therefore be stored the first time and reused at every later step; that store is the KV cache (slide 25), and the decode step for the new token reads it rather than recomputing the whole prefix.

A detail about the diagonal: the mask allows $s = t$, so each position attends to itself. This is harmless and useful (a position's own value is a sensible default), and it guarantees that each row has at least one readable entry, so the softmax is always defined. Encoder models such as BERT use no mask at all, so every position reads every other in both directions, which is the difference drawn on slide 29.

$$M_{ts} = \begin{cases} 0 & s \le t \\ -\infty & s > t \end{cases}, \qquad \alpha_{ts} = 0 \text{ for all } s > t \text{ after the softmax}$$

one pass over $x_1,\dots,x_T$ gives T predictions $P(x_{t+1} \mid x_{\le t})$, $t = 1,\dots,T$

Terms

causal mask The additive mask that forbids attention from position t to any position $s > t$; the only thing that makes a decoder-only transformer autoregressive.

teacher forcing Training on a known sequence by feeding the true tokens as input at every position and scoring each position's prediction of the next one; the mask makes it valid in one pass.

prefix The first t tokens of a sequence; the context available to position t.

KV cache The stored keys and values of all earlier positions at every layer, reused at each generation step because the mask guarantees they never change (slide 25).

bidirectional attention Attention with no causal mask, as in BERT, where every position reads every other; suited to understanding a complete text, not to generating one.

Intuition

The mask is the rule "you may not look at the answer key." Everything an LLM does follows from being made to guess each next token honestly, billions of times, and the mask is the mechanism that keeps the guessing honest while still allowing all T guesses to be scored in one pass. When reading any later claim about efficiency (caching, prefill versus decode), ask: which part of it is the mask's guarantee that the past never changes?

5.6 Slide 15: Multi-head attention

Several small attention mechanisms run side by side, each free to track a different relationship.

What the slide shows The (T,d) input X on the left feeds three parallel heads, drawn as boxes labelled head 1, head 2, head 3, each with its own $W_Q^{(n)}$, $W_K^{(n)}$, $W_V^{(n)}$ ("own weights, $d_k = d/h$"), each producing its own 5×5 attention pattern ("own pattern") and its own (T,d_k) output. The three patterns differ: head 1 is a sharp previous-token diagonal, head 2 spreads evenly over the past, head 3 is the running example's pattern. A note says there are h heads in total. The three outputs are concatenated into a (T,d) matrix, pass through W_O ("mix heads") and are added to the stream. Three notes: parallel views, with one head following syntax while another tracks a name; same cost, since h heads of width d/h cost about as much as one head of width d; and GPT-2 small has $d = 768$, $h = 12$, so each head works in $d_k = 64$ dimensions.

Details A single attention head produces one weight distribution per query, so it can do one kind of lookup at a time: a position can either read its predecessor or find its antecedent, not both in the same head. Multi-head attention runs h independent copies of the mechanism side by side, each with its own projection matrices and therefore its own notion of what to look for, and lets each produce its own output. Head n computes

$$Z^{(n)} = \text{softmax}\Big(\frac{Q^{(n)}K^{(n)\top}}{\sqrt{d_k}} + M\Big)V^{(n)}, \qquad Q^{(n)} = XW_Q^{(n)\top}, \text{ etc.},$$

with $d_k = d/h$. The h outputs, each $T \times d_k$, are concatenated along the feature axis into a $T \times d$ matrix and multiplied by the output projection $W_O \in \mathbb{R}^{d\times d}$, which mixes the heads' contributions and is the vector added to the residual stream. W_O is where a head's finding gets written into the shared coordinate system of the stream; without it each head would own a fixed slice of the stream's coordinates.

The cost argument on the slide is why heads are nearly free. Each head's three projections are $d \times d_k$ matrices, so all h heads together hold $3h\,d\,d_k = 3d^2$ weights, the same as one full-width head; W_O adds d^2, for the $4d^2$ per block quoted on slide 21. The score computation is h matrices of size $T \times T$ each costing $T^2 d_k$, total $T^2 d$, again the same as one wide head. The only thing that changes is that each head's dot products

live in d_k dimensions rather than d, so each head has a lower-dimensional, coarser notion of similarity, in exchange for there being h of them. Empirically that trade is strongly favourable: a few dozen specialised 64- or 128-dimensional lookups beat one 768-dimensional lookup.

GPT-2 small has $h = 12$ heads of width 64; GPT-3 has 96 heads of width 128; Llama 3 70B has 64 query heads of width 128 (slide 22). Head width has stayed at 64 to 128 across model sizes while the number of heads grew with d. Grouped-query attention (slide 29) keeps many query heads but shares keys and values among groups of them, which cuts the KV cache without reducing the number of lookups.

head n: $Z^{(n)} = \text{softmax}\left(Q^{(n)}K^{(n)\top}/\sqrt{d_k} + M\right)V^{(n)} \in \mathbb{R}^{T\times d_k}$, $d_k = d/h$

$\text{MHA}(X) = \left[Z^{(1)} \mid \cdots \mid Z^{(h)}\right] W_O^\top \in \mathbb{R}^{T\times d}$

weights: $3d^2$ (all W_Q, W_K, W_V) $+ d^2$ (W_O) $= 4d^2$, independent of h

Terms

multi-head attention (MHA) Running h attention heads in parallel, each with its own W_Q, W_K, W_V of width $d_k = d/h$, concatenating their outputs and mixing them with W_O.

head One of the h parallel attention mechanisms; it has its own weights and its own attention pattern.

concatenation Placing the h head outputs side by side along the feature axis, so that h vectors of width d/h become one vector of width d.

output projection W_O The $d \times d$ matrix applied to the concatenated head outputs; it mixes the heads and writes the result into the residual stream's coordinates.

number of heads h 12 in GPT-2 small, 96 in GPT-3, 64 in Llama 3 70B; head width d/h has stayed near 64–128.

grouped-query attention (GQA) Multi-head attention in which several query heads share one key/-value head, reducing the KV cache (slide 29; Ainslie et al. 2023).

Intuition

One head is one question per position; h heads are h questions asked at once, each answered separately, with W_O combining the answers. Splitting the width among heads is like giving a committee of twelve specialists 64 dimensions each instead of one generalist 768: the specialists are individually cruder but can look for twelve different things. When a model seems to need to track several relations simultaneously (subject, tense, the item in a list), the heads are where that parallelism lives.

5.7 Slide 16: What individual heads learn to do

Idealized attention patterns reported in interpretability studies. Rows are readers, columns are the positions they read.

What the slide shows Four 8×8 attention patterns, lower-triangular, drawn as heat maps with a caption each. *Previous token*: a bright band one step below the diagonal, each position reading the one just before it. *Local window*: a decaying blend of the last few tokens, brightest at the diagonal and fading leftward. *Induction*: over the token sequence A B C D A B C D, the second A reads the first B, the second B reads the first C, and so on; "on a repeat, read what followed last time and copy it." *Reference*: the row for *she* reads the column for *Alice*, and the other rows put most of their weight on the first column; "a pronoun reads its antecedent; idle rows park on the first token." A footer gives the sources: Clark et al. 2019 (BERT heads for

syntax and coreference), Elhage et al. 2021 and Olsson et al. 2022 (previous-token and induction heads), with the caveat that real heads are messier and many have no single clean role.

Details Nothing in the architecture assigns a job to a head; the jobs on the slide are what researchers found when they looked at trained models, and the four patterns are idealised drawings of real findings.

Previous-token heads attend from each position to the one before it, which gives every position direct access to the immediately preceding token (useful for bigram-like statistics and as a building block for the induction mechanism). *Local-window heads* average over a few recent positions, which is a learned version of a convolution; both patterns are positional rather than content-driven, and with RoPE they are easy to express since the score depends on $m - n$. Clark et al. (2019) found both in BERT, along with heads attending to specific syntactic relations (a verb's direct object, a noun's determiner) with accuracy well above chance.

Induction heads (Elhage et al. 2021; Olsson et al. 2022) implement a copying rule: when the current token A has occurred before, look at the token that followed the earlier occurrence and predict it will follow again. Mechanically this takes two heads in two layers: a previous-token head in an earlier layer writes "the token before me was A" into each position's stream, and the induction head in a later layer uses a query of "I am A" against keys of "the token before me was A" to find the position right after the earlier A, then copies that position's token into the prediction. On the slide's sequence A B C D A B C D, the second A reads the first B, and so on down the diagonal offset. Olsson et al. argued that induction heads are the main mechanism of in-context learning in small models and that their formation coincides with a visible bend in the training loss curve.

Reference heads are the attention of slide 10: a pronoun's row reads its antecedent. The same slide also shows what the other rows do when they have nothing to find: they put their weight on the first token. Clark et al. noticed heads in BERT attending heavily to the separator token when they had nothing better to do and read it as a no-op; in decoder models the same role falls to the first position, later named the *attention sink*. Because the softmax must put its weight somewhere, a position that has been trained to carry a near-zero value becomes the default dump, and the resulting large weights on the first token are an artefact of normalisation rather than a sign of importance. This matters for efficiency work: removing the first token from the cache breaks a model, and keeping it is cheap.

The caveat in the footer deserves emphasis. The four pictures are the clean cases. Many heads mix several patterns, change behaviour with the input, or do nothing identifiable; some are redundant and can be removed with little effect; and the division of labour among heads differs from model to model. The point of the slide is that attention patterns are legible enough to study, not that every head has a name.

Terms

previous-token head A head whose pattern is a diagonal one step below the main diagonal: each position reads its predecessor.

local-window head A head that spreads its weight over a few recent positions, like a short moving average.

induction head A head that, on a repeated token, attends to the token that followed the earlier occurrence and copies it; the mechanism of simple in-context pattern completion (Elhage et al. 2021; Olsson et al. 2022).

in-context learning A model's ability to pick up a pattern from examples in the prompt without any change to its weights.

attention sink A position, usually the first token, that receives large attention weight from rows that have nothing to find; a consequence of the softmax having to put its weight somewhere.

interpretability The study of what trained networks compute internally, for example by reading attention

patterns or tracing circuits of heads.

circuit A small set of heads and MLP units across layers that together implement an identifiable computation, such as induction.

Intuition

Attention patterns are the one part of a transformer you can look at directly and sometimes read like a diagram, which is why so much interpretability work starts there. The picture to keep: a head is a wire that can be switched by content, and a few wires in sequence (previous-token, then induction) form a circuit that does something a single layer could not. The question to ask of any claimed head role is whether it survives on inputs the researchers did not pick.

6 MLP (slides 17–18)

6.1 Slide 17: The MLP: where each token is transformed

A two-layer network applied to every position separately: widen, apply a nonlinearity, project back.

What the slide shows A small fully connected network drawn left to right: three grey input units ("d in"), twelve hidden units in the middle ("$4d$ hidden units"), three blue output units ("d out"), with every input wired to every hidden unit and every hidden unit to every output. The hidden units are shaded by activation, and most are white: "nonlinearity σ: many units stay at zero." The first set of connections is labelled "W_1: expand", the second "W_2: project back". On the right, the formula $\mathrm{MLP}(x) = W_2\,\sigma(W_1 x + b_1) + b_2$ and four notes: the same weights are applied at every position and no information moves between tokens; the hidden width is $d_{\mathrm{ff}} = 4d$, in GPT-2 small $768 \to 3072 \to 768$; the MLP holds most of the weights, $8d^2$ per block against $4d^2$ for attention, about two thirds; and one reading of it is as a key–value memory, in which rows of W_1 detect a pattern in the stream and the matching columns of W_2 write the associated update (Geva et al. 2021).

Details The second sublayer of every block is the simplest neural network there is: a linear map up to a wider space, an elementwise nonlinearity, and a linear map back down. For a single position's vector $x \in \mathbb{R}^d$,

$$\mathrm{MLP}(x) = W_2\,\sigma(W_1 x + b_1) + b_2, \qquad W_1 \in \mathbb{R}^{d_{\mathrm{ff}} \times d},\ W_2 \in \mathbb{R}^{d \times d_{\mathrm{ff}}},$$

with $d_{\mathrm{ff}} = 4d$ in the 2017 design and in GPT-2 (the ratio 4 is a convention that has stuck, though gated variants change it; next slide). The same W_1, W_2 are applied at every position of the sequence, which is why the paper called it the "position-wise feed-forward network": it is a function of one token's vector at a time, and in matrix form $\mathrm{MLP}(X) = \sigma(XW_1^\top + b_1)W_2^\top + b_2$ simply applies it to each row. No row influences any other. The MLP therefore cannot do what attention does, and attention cannot do what the MLP does: attention moves information between positions but is linear in the values it moves (the weights are nonlinear, the mixing is not), while the MLP applies a genuine nonlinear transformation to each position's accumulated information.

The weight count is where the MLP dominates. W_1 and W_2 each have $d \cdot 4d = 4d^2$ entries, so the MLP has $8d^2$ weights per block against attention's $4d^2$; of the $12d^2$ per block, two thirds are in the MLP. For GPT-2 small that is $8 \cdot 768^2 = 4.72$M in each of 12 MLPs, 45.5% of the whole model (slide 27). The same two-thirds share holds for every dense model with $d_{\mathrm{ff}} = 4d$, and in large models, where the blocks are nearly everything, the MLPs are nearly two thirds of all parameters. The compute follows the weights: each position passes through $8d^2$ multiply–adds in the MLP, linear in T, with no T^2 term.

The key–value reading (Geva et al. 2021) explains what all those weights are for. Write the hidden activation as $\sigma(W_1 x)$: row r of W_1 is a vector k_r in the stream's space, and the rth hidden unit is $\sigma(k_r \cdot x)$, which is large only when the stream vector matches the pattern k_r. Column r of W_2 is a vector v_r, and the output is $\sum_r \sigma(k_r \cdot x)\, v_r$: a weighted sum of the "value" vectors v_r, weighted by how well the input matched each "key" k_r. The MLP is a memory with d_{ff} entries, read by pattern match, with the nonlinearity as the switch that keeps non-matching entries silent. Geva et al. found that many keys in trained models respond to human-interpretable patterns (a topic, a syntactic construction) and the corresponding values push the next-token distribution toward particular tokens; later work located much of a model's factual recall in these layers. The slide's mostly-white hidden units are this picture: for any given input, only a few keys match, and the rest contribute nothing.

$\text{MLP}(x) = W_2\,\sigma(W_1 x + b_1) + b_2, \quad d \to d_{\text{ff}} = 4d \to d$

weights: $2 \cdot d \cdot d_{\text{ff}} = 8d^2$ (plus $5d$ biases); GPT-2 small: $768 \to 3{,}072 \to 768$, 4,722,432 per block

key–value form: $\text{MLP}(x) = \sum_{r=1}^{d_{\text{ff}}} \sigma(k_r \cdot x)\, v_r$, k_r = row r of W_1, v_r = column r of W_2

Terms

MLP (feed-forward network) The position-wise two-layer network in each block: expand to d_{ff}, apply a nonlinearity, project back to d. Also called the FFN.

hidden width d_{ff} The width of the MLP's middle layer; $4d$ in the original design and GPT-2.

nonlinearity (activation function) The elementwise function σ between the two matrices; without it the two linear maps would compose into one.

hidden unit One coordinate of the middle layer; in the key–value reading, one memory entry with key k_r and value v_r.

key–value memory (MLP reading) The interpretation of W_1 rows as patterns to detect and W_2 columns as updates to write, so that the MLP is an associative memory read by matching (Geva et al. 2021).

bias The vectors b_1, b_2 added after each matrix; present in GPT-2, dropped in most recent models.

Intuition

If attention is where a token asks questions of the others, the MLP is where it consults its own memory of the world. Each hidden unit is a card in a card index: a condition on the front (the key), an update to make on the back (the value). A token's vector flips through all d_{ff} cards at once, and the few that match contribute their updates. That is why most of the weights are here: the cards are the model's stored knowledge, and more cards store more.

6.2 Slide 18: Activation functions

The nonlinearity between the two matrices. Without it, the whole MLP would collapse into one linear map.

What the slide shows A plot of three activation functions over $x \in [-4, 3]$: ReLU as a dashed grey line (zero for negative x, then the identity), GELU in blue and SiLU in pink, both smooth, both dipping slightly below zero for small negative inputs before approaching the identity. A caption notes that the dip means small negative inputs still pass some gradient. On the right, the three definitions with their users: ReLU $= \max(0, x)$, the original Transformer (2017); GELU $= x\Phi(x)$ with Φ the normal CDF, used by BERT and GPT-2; SiLU $= x\,\text{sigmoid}(x)$, the gate inside SwiGLU. Below, SwiGLU, the gated MLP of Llama and Mistral: $\text{MLP}(x) = W_2\big(\text{SiLU}(W_1 x) \odot W_3 x\big)$, three matrices with hidden width $\frac{8}{3}d$ to keep the same $8d^2$ weight budget

(Shazeer 2020).

Details The subtitle states the reason a nonlinearity is needed at all: $W_2(W_1x) = (W_2W_1)x$ is a single linear map of rank at most d, so without σ the MLP would be a $d \times d$ matrix, a tenth of the weights for none of the expressive power. The elementwise function in between breaks the composition. Any non-polynomial function would do in principle; the three on the slide are the ones that have been used in practice.

ReLU, $\max(0,x)$, is the simplest: it zeroes negative inputs and passes positive ones unchanged. Its gradient is 0 or 1, cheap and well-behaved, and it produces exactly sparse activations (the white units of the previous slide). Its weakness is that a unit whose input is negative receives no gradient at all and can stop learning. GELU (Hendrycks and Gimpel 2016) replaces the hard switch by a smooth one: $x\Phi(x)$, where Φ is the standard normal cumulative distribution function, so the input is multiplied by the probability that a standard normal variable is below it. For large positive x this is x; for large negative x it is 0; near zero it is smooth and dips to about -0.17 at $x \approx -0.75$. GPT-2 uses the tanh approximation $0.5x\big(1+\tanh(\sqrt{2/\pi}\,(x+0.044715x^3))\big)$, which is what the slide plots. SiLU, $x\,\mathrm{sigmoid}(x)$, is the same idea with the logistic function in place of Φ (proposed in the same GELU paper and independently as Swish by Ramachandran et al. 2017); it dips to about -0.28 at $x \approx -1.28$. The two smooth functions behave almost identically in practice, and their advantage over ReLU is modest but consistent.

SwiGLU changes the structure rather than the curve. A gated linear unit (Dauphin et al. 2017) multiplies two linear projections of the input elementwise, one of them passed through a gate function; Shazeer (2020) tried several gates in the transformer MLP and found the SiLU-gated version best. The MLP becomes

$$\mathrm{MLP}(x) = W_2\big(\mathrm{SiLU}(W_1x) \odot W_3x\big),$$

with three matrices instead of two: W_1 produces the gate, W_3 the signal, and their product is projected back by W_2. To keep the weight count at $8d^2$ with three matrices, the hidden width is reduced from $4d$ to $\frac{8}{3}d$ (three matrices of $d \times \frac{8}{3}d$ give $8d^2$). In practice the width is rounded to a hardware-friendly multiple: Llama 7B uses 11,008 for $d = 4{,}096$ (about $2.7d$), and later Llama models use $3.5d$ (14,336 for Llama 3 8B), spending a little more on the MLP. In the key–value reading, the gate is still a pattern detector, but the value written is now scaled by a second linear function of the input rather than being a fixed vector, which gives each memory entry more flexibility. Llama, Mistral, Qwen and most open models since 2023 use SwiGLU; GPT-2-style models and many others use GELU; the original ReLU survives in some efficiency-oriented designs because exact zeros can be skipped.

$\mathrm{ReLU}(x) = \max(0,x)$; $\mathrm{GELU}(x) = x\Phi(x) \approx 0.5x\big(1+\tanh(\sqrt{2/\pi}\,(x+0.044715x^3))\big)$; $\mathrm{SiLU}(x) = x/(1+e^{-x})$

SwiGLU MLP: $W_2\big(\mathrm{SiLU}(W_1x) \odot W_3x\big)$, $W_1, W_3 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$, $d_{\mathrm{ff}} = \frac{8}{3}d \Rightarrow 3dd_{\mathrm{ff}} = 8d^2$

Terms

ReLU Rectified linear unit, $\max(0,x)$; the activation of the 2017 Transformer.

GELU Gaussian error linear unit, $x\Phi(x)$ with Φ the standard normal CDF; smooth, used by BERT and GPT-2 (Hendrycks and Gimpel 2016).

SiLU (Swish) Sigmoid linear unit, $x\,\mathrm{sigmoid}(x)$; the gate function in SwiGLU.

sigmoid The logistic function $1/(1+e^{-x})$, rising smoothly from 0 to 1.

gated linear unit (GLU) An MLP layer computing $\sigma(W_1x) \odot W_3x$: one projection gates another elementwise (Dauphin et al. 2017).

SwiGLU The GLU with a SiLU gate, used as the MLP of Llama, Mistral and most recent open models

(Shazeer 2020).

elementwise product $\odot$ Multiplying two vectors coordinate by coordinate.

dead unit A ReLU unit whose input is negative for all inputs, so it outputs zero and receives no gradient; smooth activations avoid the hard version of this.

Intuition

The nonlinearity is a switch, and the choice among ReLU, GELU and SiLU is a choice of how softly the switch turns on. A hard switch gives exact zeros and clean sparsity; a soft one lets a unit that is slightly "off" still report how close it was, which helps learning. SwiGLU is a different idea: instead of one switch per unit, it uses one projection to decide *how much* of another projection to let through, so the value stored in each memory entry can itself depend on the input.

7 Norm and residual (slides 19–20)

7.1 Slide 19: Residual connections and where the norm goes

Each sublayer adds a correction to its input. Norm placement decides how clean the identity path is.

What the slide shows Two wiring diagrams, each a thick horizontal arrow for the main path with a branch below it. *Post-norm* (original, 2017): the branch leaves the main path, passes through F (attention or MLP), rejoins at a $+$ node, and the sum then passes through a Norm box sitting *on* the main path; equation $x_{l+1} = \mathrm{Norm}(x_l + F(x_l))$, "the norm sits on the main path". *Pre-norm* (GPT-2 onward; the standard today): the branch passes through Norm and then F, and rejoins at $+$; the main path, shaded as an "untouched identity path", never passes through a norm; equation $x_{l+1} = x_l + F(\mathrm{Norm}(x_l))$, "the norm sits inside the branch". Two notes: residuals give signal and gradient a direct route through all L layers, which is what makes very deep stacks trainable (He et al. 2016); pre-norm is more stable at depth with little or no learning-rate warm-up (Xiong et al. 2020) and needs one final norm before the output head.

Details Two ideas are on this slide, and they are often confused because they appear together.

The first is the residual connection. Instead of computing $x_{l+1} = F(x_l)$, each sublayer computes $x_{l+1} = x_l + F(x_l)$: its input is passed through unchanged and the sublayer's output is added to it. The sublayer learns a *correction*, and a sublayer that has learned nothing ($F \approx 0$) is harmless rather than destructive. He et al. (2016) introduced the idea for image networks, where it let depth go from about twenty layers to over a hundred, and the reason is in the gradient: by the chain rule, $\partial x_{l+1}/\partial x_l = I + \partial F/\partial x_l$, so the gradient of the loss with respect to any earlier layer contains a direct term that is not multiplied by anything, however deep the stack. Without the I, a gradient passing through L layers is a product of L Jacobians and shrinks or explodes exponentially in L. With it, the stack is in effect a sum of shallow paths, and the lanes of slide 4 are this structure drawn out: the residual stream is the identity path, and every sublayer writes into it.

The second idea is where normalization sits relative to that addition. The 2017 design applied the norm *after* the addition, $\mathrm{Norm}(x + F(x))$, so the main path itself passes through a norm at every sublayer: the identity path is rescaled $2L$ times on the way through, and the gradient's direct term is multiplied by the norm's Jacobian each time. Xiong et al. (2020) showed that at initialisation this makes the gradients of the last layers large and the whole model sensitive to the learning rate, which is why the original recipe needed a careful warm-up period of small learning rates and still became unstable when the stack was deep. The alternative, moving the norm to the start of the branch, $x + F(\mathrm{Norm}(x))$, leaves the identity path truly untouched, gives well-scaled gradients at every depth, and trains with little or no warm-up. GPT-2 adopted it

(its paper describes moving the layer norm to the input of each sub-block and adding a final one after the last block), and every major model since has used it. One consequence is on the slide: because the stream itself is never normalised, its scale grows with depth, and a final norm must be applied before the unembedding (slide 23).

The deck's block (slide 21) is pre-norm: each sublayer is "norm, compute, add". A few recent models add a second norm on the branch output before the addition (sometimes called sandwich norm) or other variants to tame the growth of the stream's scale, but the pre-norm residual block is the baseline against which they are measured.

post-norm (2017): $x_{l+1} = \mathrm{Norm}\left(x_l + F(x_l)\right)$ pre-norm (GPT-2 onward): $x_{l+1} = x_l + F\left(\mathrm{Norm}(x_l)\right)$

residual gradient: $\dfrac{\partial x_{l+1}}{\partial x_l} = I + \dfrac{\partial F}{\partial x_l}$ (pre-norm), so the loss gradient reaches every layer through an unattenuated path

Terms

post-norm Applying the normalization after the residual addition, on the main path; the 2017 arrangement.

pre-norm Applying the normalization at the entrance of the sublayer branch, leaving the main path unnormalised; the arrangement of GPT-2 and all current LLMs.

identity path The route through the stack along which a position's vector passes unchanged except for additions; what the residual connections create.

Jacobian The matrix of partial derivatives of a layer's output with respect to its input; the chain rule multiplies Jacobians along a path.

vanishing or exploding gradients The shrinking or growth of gradients through many layers when each layer multiplies them by a factor; residual connections prevent it.

learning-rate warm-up Starting training with a very small learning rate and raising it over the first few thousand steps; needed by post-norm transformers, largely optional with pre-norm.

final norm The normalization applied to the stream after the last block and before the unembedding; required in pre-norm models because the stream itself is never normalised.

Intuition

The residual stream is a conveyor belt, and each sublayer is a worker who may add something to what passes by but may not take the item off the belt. Post-norm puts a scale under the belt at every station and reweighs everything that passes; pre-norm lets each worker weigh a copy before deciding what to add, and leaves the belt alone. The belt is why a hundred stations can be strung together, and leaving it alone is why they can be trained without coaxing.

7.2 Slide 20: LayerNorm and RMSNorm

Rescale each token's vector to a standard size, using only that token's own d features.

What the slide shows Three bar charts of an eight-dimensional vector. In the middle, the input $x = (2.6, -0.4, 1.2, 3.8, 0.6, -1.4, 2.2, 1.0)$ for one token with $d = 8$, with a dashed line at its mean $\mu = 1.2$ and the statistics $\mu = 1.2$, $\sigma = 1.56$, RMS $= 1.97$. On the left, after LayerNorm ("center, scale"): $(0.90, -1.03, 0.00, 1.67, -0.38, -1.67, 0.64, -0.13)$, centred on zero with reference lines at ± 1. On the

right, after RMSNorm ("scale only"): $(1.32, -0.20, 0.61, 1.93, 0.31, -0.71, 1.12, 0.51)$, not centred. Under them the formulas $\gamma \odot \frac{x-\mu}{\sigma} + \beta$ (subtract the mean, divide by the standard deviation, learned gain and bias) and $\gamma \odot \frac{x}{\mathrm{RMS}(x)}$ (no centering, no bias, less work; Llama, Mistral, Qwen, Gemma). A note: shown with $\gamma = 1$, $\beta = 0$; the statistics come from a single token, so nothing depends on the batch or on other positions.

Details A normalization layer puts a vector onto a standard scale so that what follows sees inputs of predictable size, whatever happened before. The important design choice in the transformer is *which numbers the statistics are computed over*: LayerNorm (Ba, Kiros and Hinton 2016) uses the d coordinates of one position's vector and nothing else. The mean and variance are those of the eight bars on the slide, not of a batch of examples (batch normalization, common in vision) and not of other positions. This makes the operation position-wise, deterministic given the token's own vector, identical at training and inference, and independent of sequence length and batch size.

LayerNorm centres and scales: $\mathrm{LN}(x) = \gamma \odot (x - \mu)/\sqrt{\sigma^2 + \varepsilon} + \beta$, where $\mu = \frac{1}{d}\sum_c x_c$, $\sigma^2 = \frac{1}{d}\sum_c (x_c - \mu)^2$, ε is a small constant for numerical safety, and $\gamma, \beta \in \mathbb{R}^d$ are learned per-coordinate gain and bias, so the layer can undo the normalization on any coordinate where that is useful. On the slide's input, $\mu = 9.6/8 = 1.2$, the squared deviations sum to 19.44 so $\sigma^2 = 2.43$ and $\sigma = 1.56$, and the first coordinate becomes $(2.6 - 1.2)/1.56 = 0.90$. RMSNorm (Zhang and Sennrich 2019) drops the centering: $\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_c x_c^2 + \varepsilon}$ and $\mathrm{RMSNorm}(x) = \gamma \odot x/\mathrm{RMS}(x)$, with a gain but no bias. On the slide, the squares sum to 30.96, so $\mathrm{RMS} = \sqrt{3.87} = 1.97$ and the first coordinate becomes $2.6/1.97 = 1.32$. The output is not centred (its mean is $1.2/1.97 = 0.61$), which does not matter in practice: Zhang and Sennrich found the re-scaling to be what stabilises training, with the re-centering contributing little, and RMSNorm is cheaper (one pass over the vector instead of two, and no subtraction). Llama, Mistral, Qwen and Gemma use RMSNorm; GPT-2 and BERT use LayerNorm.

Parameter cost is negligible: $2d$ for LayerNorm, d for RMSNorm, per norm. GPT-2 small has two norms per block plus the final one, 25 in all, $25 \times 2 \times 768 = 38{,}400$ parameters, 0.03% of the model (slide 27). What the norms buy is scale control. The residual stream's magnitude grows with depth as sublayers add to it; without a norm at the entrance of each branch, the attention scores and MLP pre-activations in later blocks would see ever larger inputs, the softmaxes would saturate (slide 12) and the activations would drift out of the range in which the nonlinearities are useful. The norm resets the scale at every branch entrance without touching the stream itself.

$$\mu = \tfrac{1}{d}\textstyle\sum_c x_c,\ \ \sigma^2 = \tfrac{1}{d}\textstyle\sum_c (x_c - \mu)^2,\ \ \mathrm{RMS}(x) = \sqrt{\tfrac{1}{d}\textstyle\sum_c x_c^2}$$

LayerNorm: $\gamma \odot \dfrac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta$ ($2d$ parameters) RMSNorm: $\gamma \odot \dfrac{x}{\mathrm{RMS}(x)}$ (d parameters)

slide's example: $\mu = 1.2$, $\sigma = 1.56$, $\mathrm{RMS} = 1.97$; $2.6 \mapsto 0.90$ (LN), $2.6 \mapsto 1.32$ (RMS)

Terms

normalization layer An operation that rescales a vector to a standard size using statistics of its own coordinates, then applies a learned gain (and possibly bias).

LayerNorm Normalization that subtracts the mean and divides by the standard deviation of one position's d coordinates, then applies gain γ and bias β (Ba, Kiros and Hinton 2016).

RMSNorm Normalization that divides by the root-mean-square of the coordinates with no centering and no bias, only a gain γ (Zhang and Sennrich 2019).

root mean square (RMS) $\sqrt{\frac{1}{d}\sum_c x_c^2}$, the typical magnitude of a coordinate.

gain and bias γ, β The learned per-coordinate scale and shift applied after normalization, so the layer can restore any scale that the following computation prefers.

epsilon ε A small constant (typically 10^{-5} or 10^{-6}) added inside the square root so that a near-zero vector does not produce a division by zero.

batch normalization Normalization using statistics across a batch of examples; common in vision, unsuitable for sequence models and not used in transformers.

Intuition

A norm is a volume knob that is turned, for every token separately, so that the signal entering a sublayer always has the same loudness. The content, which direction the vector points, is kept; only its length (and, in LayerNorm, its offset) is reset. The learned gain lets the model decide that some coordinates should be louder than others. The reason it uses only the token's own coordinates is that anything else would make a token's representation depend on who it shares a batch with, which is both a nuisance at inference and a leak of information between examples.

8 Block and stack (slides 21–22)

8.1 Slide 21: One complete transformer block

Two sublayers, each wrapped as norm, compute, add. The output has the same shape as the input.

What the slide shows A thick arrow from x_l on the left to x_{l+1} on the right, both of shape (T, d). Two branches hang below it. The first: a Norm box, then a large orange Multi-head attention box (showing Q, K, V cells and a small causally masked grid, "h heads, causal mask"), rejoining the main path at a $+$; caption "tokens exchange information". The second: a Norm box, then a blue MLP box ("$d \to 4d \to d$"), rejoining at a second $+$; caption "each token is transformed". On the right, the equations $h = x_l + \mathrm{Attn}(\mathrm{Norm}(x_l))$ and $x_{l+1} = h + \mathrm{MLP}(\mathrm{Norm}(h))$ and the weight count: about $12d^2$ per block, $4d^2$ in attention (W_Q, W_K, W_V, W_O) and $8d^2$ in the MLP (W_1, W_2); GPT-2 small, 7.1M per block, 12 blocks. A footer: nothing here depends on T, so the same weights serve any sequence length, and same-shape input and output is what lets blocks stack.

Details This slide assembles slides 10 through 20 into the unit that is repeated. A block takes the stream $X_l \in \mathbb{R}^{T \times d}$ and returns X_{l+1} of the same shape by two residual updates:

$$H = X_l + \mathrm{MHA}\big(\mathrm{Norm}_1(X_l)\big), \qquad X_{l+1} = H + \mathrm{MLP}\big(\mathrm{Norm}_2(H)\big).$$

Each sublayer reads a normalised copy of the stream, computes something, and adds it back; the stream itself is only ever added to. Attention is the sublayer where rows of X interact, under the causal mask; the MLP is the sublayer where each row is transformed on its own. The two norms have separate gains.

The weight count per block is the sum of its parts. Attention: W_Q, W_K, W_V together $3d^2$ and W_O another d^2, total $4d^2$, however many heads. MLP: W_1 and W_2 at $4d^2$ each, total $8d^2$. Norms: $2d$ each, negligible. So a block has about $12d^2$ weights, and the ratio two-to-one between MLP and attention holds for any model with $d_{\mathrm{ff}} = 4d$. GPT-2 small, $d = 768$: $12 \times 768^2 = 7{,}077{,}888$, plus biases ($4d$ in attention, $5d$ in the MLP) and norm parameters, giving the 7,084,800 per block that slide 27 counts as attention plus MLP, or 7.1M; twelve blocks make 85M of the model's 124M. With SwiGLU the MLP has three matrices of $d \times \frac{8}{3}d$, still $8d^2$ by design, so the $12d^2$ rule survives in Llama-style models up to the rounding of d_{ff}.

The footer makes the point that lets everything else work. None of the weights has a dimension that depends on T: W_Q is $d \times d$ whether the sequence has 5 tokens or 5,000, the MLP acts on one row at a time,

the norms likewise. The only T-dependence is in the attention matrix, which is computed, not stored as a weight. A trained block can therefore be applied to any length of sequence (up to the limit of its position encoding), which is what makes the context length a run-time quantity rather than an architectural one. And because the output shape equals the input shape, X_{l+1} can be handed to an identical block with its own weights, which is the stack of the next slide.

$H = X_l + \mathrm{MHA}(\mathrm{Norm}_1(X_l)), \quad X_{l+1} = H + \mathrm{MLP}(\mathrm{Norm}_2(H)), \quad$ all $T \times d$
weights per block $\approx 4d^2 + 8d^2 = 12d^2$; GPT-2 small: 7,084,800 (with biases) $\approx$ 7.1M, $\times$12 blocks

Terms

transformer block The repeated unit: a pre-norm attention sublayer followed by a pre-norm MLP sublayer, each with a residual addition; input and output are both $T \times d$.

$12d^2$ rule The weight count of one block with $d_{\mathrm{ff}} = 4d$ (or SwiGLU at $\frac{8}{3}d$): $4d^2$ in attention, $8d^2$ in the MLP, biases and norms aside.

shape preservation The property that a block's output has the same shape as its input, so blocks compose and the depth L is a free choice.

length independence The property that no weight's size depends on T, so one set of weights serves any sequence length.

norm gains (per block) The two separate γ (and β) vectors of a block's two norms; the only parameters outside the four attention and two MLP matrices.

Intuition

A block is a pair of "read, think, write back" steps on a shared document: first every position reads what the others wrote (attention), then every position revises its own line (MLP), and in both cases the revision is appended rather than overwritten. The shapes are the contract: as long as the document keeps its $T \times d$ shape, any number of such editing passes can follow, and none of them needs to know how long the document is.

8.2 Slide 22: Stacking blocks: depth and width

The same block repeated L times with separate weights. Later blocks build on what earlier blocks wrote to the stream.

What the slide shows On the left, a vertical stack: an Embed + position plate at the bottom, then block 1, block 2, dots, block $L-1$, block L, each drawn as an Attention half and an MLP half, then a Norm + unembed plate at the top, with an arrow running up through all of it. On the right, a table of six models: GPT-2 small (2019) with $L = 12$, $d = 768$, 12 heads, 124M parameters; GPT-2 XL (2019) 48, 1,600, 25, 1.56B; GPT-3 davinci (2020) 96, 12,288, 96, 175B; Llama-2 7B (2023) 32, 4,096, 32, 6.7B; Llama-3 70B (2024) 80, 8,192, 64, 70.6B; Llama-3.1 405B (2024) 126, 16,384, 128, 406B. Two notes: the scaling rule, block weights grow as $12Ld^2$, so doubling the depth doubles them and doubling the width quadruples them; and the division of labour, a tendency from probing studies, that earlier layers carry more local and syntactic features and later layers more abstract ones (Tenney et al. 2019).

Details The model is L copies of the block, each with its own weights, applied in sequence to the stream, between the embedding at the bottom and the output head at the top. Because each block adds to the stream rather than replacing it, block l sees the embedding plus everything blocks 1 to $l-1$ wrote, and can read features that earlier blocks computed. Depth is composition: a two-layer circuit like induction (slide 16)

needs a previous-token head in one block and an induction head in a later one, and more elaborate behaviours need more.

The table shows how the two numbers have moved. Width d grew from 768 to 16,384, a factor of 21; depth L from 12 to 126, a factor of 10; and the parameter count, which goes as $12Ld^2$ for the blocks, from 124M to 406B, a factor of about 3,300. The number of heads has tracked d so that the head width stays at 64 (GPT-2) to 128 (GPT-3 and Llama). For the three Llama models the $12Ld^2$ rule gives 6.4B, 64B and 406B against 6.7B, 70.6B and 406B actual (the differences are the embedding and output tables, the $3.5d$ SwiGLU width, and the GQA reduction in W_K, W_V); for GPT-3, $12 \times 96 \times 12{,}288^2 = 174$B against 175B. The rule is a good estimate of any dense transformer's size from two numbers.

How to split a budget between depth and width is an empirical question. Kaplan et al. (2020) found that, at fixed parameter count, loss depends only weakly on the ratio of width to depth over a wide range, which is why the table's aspect ratios vary (GPT-3's $d/L = 128$, Llama 3 70B's 102, GPT-2 small's 64) without any of them being wrong. Deeper is somewhat more expressive per parameter but harder to parallelise and slower per token, since the blocks run in sequence; wider gives larger matrix products that use accelerators more efficiently. Modern models have settled on roughly $d/L \approx 100$.

The division-of-labour note is a finding about what the stack does with its depth. Tenney, Das and Pavlick (2019) probed BERT's layers for the classical NLP pipeline (part of speech, parsing, named entities, semantic roles, coreference) and found that the information needed for each task is most accessible at increasing depth, in roughly that order: surface and syntactic features early, semantic and discourse features late. Later studies of decoder models find a similar picture, with early layers resolving token identity and local syntax, middle layers doing most of the factual recall in their MLPs, and the last layers converting the stream into a next-token prediction. It is a tendency, not a rule; the layers are not assigned tasks, and the boundaries are soft.

block parameters $\approx 12Ld^2$; total $\approx 12Ld^2 + 2Vd$ (separate embedding and unembedding) or $12Ld^2 + Vd$ with $W_U = E^\top$ (tied), plus position and norm parameters

GPT-3: $12 \times 96 \times 12{,}288^2 \approx 1.74 \times 10^{11}$; Llama-2 7B: $12 \times 32 \times 4{,}096^2 \approx 6.4 \times 10^9$

Terms

stack The sequence of L blocks applied one after another, between the embedding and the output head.

$12Ld^2$ **rule** The estimate of a dense transformer's block parameters from its depth and width; the embedding tables add Vd (or $2Vd$ untied).

aspect ratio The ratio d/L of width to depth; models in use range from about 60 to 130 with similar loss at equal size.

probing Training a small classifier on a frozen model's internal vectors to test what information they contain at each layer.

classical NLP pipeline The traditional sequence of language-processing stages (part-of-speech tagging, parsing, entity recognition, semantic roles, coreference) that Tenney et al. found reflected in the order of BERT's layers.

layer specialisation The tendency for earlier layers to hold local and syntactic features and later layers more abstract ones; a soft trend rather than a fixed assignment.

Intuition

Stacking is what turns a lookup-and-rewrite step into reasoning: each block can only do one round of reading and one round of rewriting, so anything that needs several rounds needs several blocks. The $12Ld^2$ rule is worth memorising because it answers "how big is this model?" from the two numbers every model card prints. And the probing result is best read as a diagnostic, not a design principle: nobody told the early layers to do syntax; it is what falls out of needing syntax first.

9 Output (slides 23–25)

9.1 Slide 23: The output head: from a vector to a distribution

The final vector at the last position is compared with every vocabulary entry, then normalized.

What the slide shows On the left, the final vector h_T of width d (eight grey cells), passing through a Norm box and then an Unembed box labelled $W_U : d \to V$. On the right, two bar charts over the six-token toy vocabulary. The first, "logits z: any real number", shows *mat* 5.10, *floor* 4.32, *sofa* 3.76, *roof* 3.06, *bed* 2.90, *moon* 0.50. An arrow labelled softmax leads to the second, "probabilities: sum to 1": .507, .233, .133, .066, .056, .005. A note says a real model scores all V entries. Below, the equations $z = W_U \,\mathrm{Norm}(h_T) \in \mathbb{R}^V$ and $p_i = e^{z_i} / \sum_j e^{z_j}$, and two notes: z_i is the dot product of the final vector with token i's row of W_U, one logit per token; and weight tying, where GPT-2 and others reuse the embedding matrix, $W_U = E^\top$ (Press and Wolf 2017).

Details After the last block, the stream at position T is a vector $h_T \in \mathbb{R}^d$ that has gathered, through L rounds of attention and MLP, whatever the model has concluded about what comes next. The output head converts it into a distribution in two steps. First the final norm (slide 19), required because the pre-norm stream is never normalised on its own. Then the unembedding: a matrix $W_U \in \mathbb{R}^{V \times d}$ whose row i is a vector for token i, so that $z = W_U \,\mathrm{Norm}(h_T)$ is a vector of V scores, z_i being the dot product of the final vector with token i's row. These scores are the *logits*: unnormalised, any real number, with only differences between them meaningful. The softmax turns them into probabilities. On the slide, the exponentials of $(5.10, 4.32, 3.76, 3.06, 2.90, 0.50)$ are $(164.0, 75.2, 42.9, 21.3, 18.2, 1.6)$, their sum is 323.3, and dividing gives $(0.507, 0.233, 0.133, 0.066, 0.056, 0.005)$. The gap of 0.78 between the top two logits becomes a probability ratio of $e^{0.78} = 2.2$; the 4.6-logit gap to *moon* becomes a ratio of 100.

The unembedding is the mirror image of the embedding: E maps a token to a vector, W_U maps a vector back to a score for each token, and both have Vd entries. Press and Wolf (2017) proposed using one matrix for both, $W_U = E^\top$, so that a token's output row is its input embedding. The argument is that the two should agree anyway (a vector close to the embedding of *mat* should score *mat* highly), the saving is Vd parameters, which in small models is large (38.6M of GPT-2 small's 124M, a third), and the shared matrix gets gradient from both ends. GPT-2 ties; so do Gemma and many small models. Large models typically do not, since Vd is a negligible share of their parameters and an untied head is slightly more flexible: Llama 2 and the larger Llama 3 models keep separate matrices.

During generation only the last position's row is needed, but the same computation can be applied to every position at once: $Z = \mathrm{Norm}(X^{(L)})W_U^\top$ is $T \times V$, and during training all T rows are used (slide 26). The output head is also the largest single matrix product per token in a small model, $d \times V$ against $12d^2$ for a block: for GPT-2 small, $768 \times 50{,}257 = 38.6$M multiply–adds per token in the head against 7.1M per block, so the head costs more than five blocks.

$$z = W_U \,\mathrm{Norm}(h_T) \in \mathbb{R}^V, \quad z_i = \langle W_U[i], \mathrm{Norm}(h_T)\rangle, \quad p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

tied head: $W_U = E^\top$, saving Vd parameters; cost per position: Vd multiply–adds

slide's example: logits $(5.10, 4.32, 3.76, 3.06, 2.90, 0.50)$

probabilities $(.507, .233, .133, .066, .056, .005)$

Terms

output head The final norm, the unembedding and the softmax: the components that turn the last stream vector into a distribution over the vocabulary.

logit An unnormalised score z_i for token i, any real number; the softmax of the logits is the probability distribution.

unembedding matrix W_U The $V \times d$ matrix whose row i scores token i by a dot product with the final vector.

weight tying Using the embedding matrix as the unembedding, $W_U = E^\top$, so that one $V \times d$ table serves both ends (Press and Wolf 2017). GPT-2 ties; Llama does not.

final vector h_T The residual stream at the last position after the last block; the input to the output head during generation.

normalisation (of probabilities) Dividing by the sum so that the V values add to 1; what the softmax's denominator does.

Intuition

The output head is the embedding run backwards: instead of "which vector is this token?" it asks "which token is this vector closest to?" and answers with a score for every token at once. Everything the model knows about the next word is in the direction of h_T; the head's rows are the directions of the vocabulary, and the logits are the projections. Tying the weights says the answer to both questions should use the same dictionary.

9.2 Slide 24: Choosing the next token: temperature, top-k, top-p

The model's work ends at the distribution. How to draw from it is a separate, tunable choice.

What the slide shows Four bar charts of the same six-token distribution under different sampling settings. $T = 0.5$, sharper: .765, .161, .052, .013, .009, .000. $T = 1$, as trained: .507, .233, .133, .066, .056, .005. $T = 1.5$, flatter: .394, .234, .161, .101, .091, .018. $T = 1$ with top-$p = 0.9$: .540, .248, .142, .070, and *bed* and *moon* marked "removed". Three notes: temperature divides the logits by T before the softmax, with $T < 1$ sharpening, $T > 1$ flattening, and $T \to 0$ greedy; top-k keeps the k most likely tokens, renormalises and samples, a fixed-size shortlist; top-p (nucleus) keeps the smallest set whose probabilities add up to p, a shortlist that adapts to how confident the model is (Holtzman et al. 2020).

Details The model's output is a distribution; what to do with it is a decision made outside the model, at generation time, with no training involved. The simplest choices are at the two extremes. *Greedy* decoding always takes the most likely token; it is deterministic and tends to produce repetitive, generic text, because the single most likely continuation at each step is rarely the most likely continuation overall and often loops. *Pure sampling* draws from the distribution as given; it reproduces the model's own uncertainty, but the long tail of low-probability tokens (there are tens of thousands of them, each individually unlikely but collectively

not) produces occasional nonsense. The three controls on the slide sit between these.

Temperature rescales the logits before the softmax: $p_i \propto e^{z_i/T}$. Dividing by $T < 1$ widens the gaps between logits and sharpens the distribution; $T > 1$ narrows them and flattens it; as $T \to 0$ the largest logit dominates completely and sampling becomes greedy; as $T \to \infty$ the distribution becomes uniform. On the slide, at $T = 0.5$ the logits are doubled, so the 0.78 gap between *mat* and *floor* becomes 1.56 and the probability ratio goes from 2.2 to 4.8; *mat* rises from .507 to .765 and *moon* falls to a ten-thousandth. At $T = 1.5$ the ratio falls to $e^{0.52} = 1.7$ and the tail grows. Temperature is the mechanism of the "creativity" setting in most interfaces; it changes the shape of the distribution without removing anything.

Top-k (Fan et al. 2018) truncates instead: keep the k highest-probability tokens, set the rest to zero, renormalise, and sample. It removes the tail outright. Its weakness is that k is fixed while the shape of the distribution is not: when the model is confident, $k = 50$ includes dozens of tokens that should not be there; when it is uncertain among a hundred reasonable options, $k = 50$ cuts half of them. Top-p, or nucleus sampling (Holtzman et al. 2020), fixes this by truncating by cumulative probability: sort the tokens, keep the smallest prefix whose probabilities sum to at least p, and sample from that set. On the slide with $p = 0.9$ the cumulative sums are $.507, .740, .873, .939$, so four tokens are kept (the smallest set reaching 0.9), their total is .939, and renormalising gives $.540, .248, .142, .070$. A confident distribution yields a short nucleus, an uncertain one a long one. Holtzman et al. showed that beam search and greedy decoding produce text that is less varied and lower in per-token surprise than human text, while pure sampling is more surprising, and that nucleus sampling lands near the human curve. In practice the controls are combined (a temperature, then a top-p or top-k cut), and the right values depend on the use: near-greedy for code and factual answers, warmer for open-ended writing.

temperature: $p_i = \dfrac{e^{z_i/T}}{\sum_j e^{z_j/T}}$; $T \to 0$ greedy, $T \to \infty$ uniform

top-k: keep the k largest p_i, renormalise; top-p: keep the smallest set S with $\sum_{i \in S} p_i \geq p$, renormalise

slide, $p = 0.9$: keep {mat, floor, sofa, roof}, total .939, $\to (.540, .248, .142, .070)$

Terms

decoding (sampling) strategy The rule used to pick a token from the model's distribution at each generation step; chosen at run time, not learned.

greedy decoding Always taking the most probable token; deterministic, prone to repetition.

temperature The divisor T applied to the logits before the softmax; below 1 sharpens the distribution, above 1 flattens it.

top-k sampling Sampling from the k most probable tokens after renormalising (Fan et al. 2018).

top-p (nucleus) sampling Sampling from the smallest set of tokens whose cumulative probability reaches p, so the shortlist adapts to the model's confidence (Holtzman et al. 2020).

renormalisation Rescaling the kept probabilities so that they sum to 1 after truncation.

beam search Keeping the B most probable partial sequences at each step and extending them; good for translation, poor for open-ended text, where it produces bland and repetitive output.

Intuition

The model proposes odds; the sampler decides how much to trust the long shots. Temperature says how seriously to take small differences in the odds; top-k and top-p say where to stop reading the list. Nucleus

sampling is the sensible default because it asks the model how long the list should be. When generated text is either dull or occasionally absurd, the sampler is the first thing to adjust, before the model.

9.3 Slide 25: Generation and the KV cache

Because of the causal mask, earlier positions never change. Their keys and values can be stored and reused.

What the slide shows Two panels, each a grid of six token columns (*The cat sat on the mat*, with *mat* the newest token in pink) by three layers. Left, "without a cache: every position is recomputed": all 18 cells are solid orange, computed at this step; the per-layer cost at step t is written as $12td^2 + 2t^2d$. Right, "with a cache: only the new position is computed": the first five columns are split into stored K (violet) and V (teal) halves, read by the new position, and only the sixth column is orange; the per-layer cost is $12d^2 + 2td$. Two notes: there are two phases, prefill, which runs the whole prompt in parallel and fills the cache, and decode, which adds one token per forward pass; and the cache size is $2LTd$ numbers, which for $L = 32$, $d = 4096$ (Llama-2 7B scale), $T = 4096$ and 16-bit floats is 2 GiB per sequence.

Details Generation runs the model once per new token, and the naive way to do it is to rerun the whole context each time. The causal mask makes that wasteful: the stream vectors, keys and values at positions $1, \ldots, t-1$ are the same at step t as they were at step $t-1$, because nothing at those positions depends on the token just added. The only genuinely new computation at step t is for position t itself, and the only thing position t needs from the past is the keys and values of the earlier positions, at every layer, to attend over. Store those once and the per-step cost collapses.

The accounting on the slide is per layer. Without a cache, step t processes all t positions: the weight products cost $12td^2$ (each position passes through $12d^2$ multiply–adds) and attention costs $2t^2d$ for the scores and mixing. With a cache, step t processes one position: $12d^2$ for its weight products, and $2td$ for its query against t cached keys and its weighted sum of t cached values. Over a generation of n tokens the uncached cost grows as n^2 in the weight term and n^3 in the attention term; the cached cost grows as n and n^2. Every serving system uses the cache.

Generation therefore has two phases with different character. *Prefill* runs the prompt's T tokens through the model in one parallel pass, exactly as in training, producing the keys and values for all of them and the first output distribution; it is compute-bound, with large matrix products. *Decode* then runs one position at a time; its matrix products are matrix-vector products that touch every weight once per token, so it is bound by memory bandwidth, not arithmetic (slide 28), and the cache read adds to the traffic.

The cache's size is what limits how many sequences a GPU can serve at once. Per layer and per position it holds a key and a value of width d (for multi-head attention, h heads of width d/h make d in all), so $2Ld$ numbers per token and $2LTd$ for a context of T. For Llama 2 7B's shape, $2 \times 32 \times 4{,}096 \times 4{,}096 = 1{,}073{,}741{,}824$ numbers; at two bytes each, 2^{31} bytes, exactly 2 GiB for one 4,096-token sequence, against 13.5 GB for the weights themselves. This is the motivation for grouped-query attention (slide 29): with h_{kv} key/value heads instead of h, the cache shrinks by h/h_{kv}; Llama 3 70B's 8 key/value heads against 64 query heads cut it eightfold.

per layer at step t: no cache $12td^2 + 2t^2d$; with cache $12d^2 + 2td$ multiply–adds

cache size: $2LTd$ numbers (MHA) $= 2LTh_{kv}d_k$ in general; Llama-2 7B shape, $T = 4{,}096$, 16-bit: 2^{31} B $= 2$ GiB

prefill: T positions in one pass (compute-bound); decode: one position per pass (bandwidth-bound)

Terms

KV cache The stored keys and values of all processed positions at every layer, reused at each decode step; $2LTd$ numbers for a context of T in a multi-head model.

prefill The first phase of generation, in which the whole prompt is processed in one parallel pass and the cache is filled.

decode The second phase, in which one new token is processed per forward pass, reading the cache and appending to it.

compute-bound Limited by arithmetic throughput; prefill and training are, because many positions share each weight read.

bandwidth-bound Limited by how fast weights and cache can be read from memory; single-sequence decode is, because each token reads every weight once for little arithmetic.

GiB 2^{30} bytes; the slide's 2 GiB is 2^{31} bytes exactly.

key/value heads h_{kv} The number of distinct key and value projections; equal to h in multi-head attention, smaller in grouped-query attention, which shrinks the cache in proportion.

Intuition

The cache is the model's notes on the conversation so far. Because nothing already written can change, the notes never need rewriting, only extending by one line per token; the price is that the notes must be kept in fast memory for as long as the conversation lasts, and at long contexts they rival the model itself in size. Prefill is reading the whole prompt at once; decode is thinking one word at a time with the notes open.

10 Training and scale (slides 26–28)

10.1 Slide 26: Training: predict every next token at once

The text supplies its own labels. One pass over a sequence of T tokens produces T predictions and T losses.

What the slide shows Five input chips *The cat sat on the* enter a box labelled "Transformer with causal mask". Above each position a small bar chart of its predicted distribution, with the bar for the correct next token in pink and its probability printed: .055, .033, .30, .67, .507. Above those, the target chips *cat sat on the mat*, shifted one position left of the inputs. At the top, the loss at each position: 2.90, 3.41, 1.20, 0.40, 0.68, with a brace giving their average, 1.72, "the number that gradient descent pushes down". On the right, the loss formula $\mathscr{L} = -\frac{1}{T}\sum_t \log p_\theta(x_{t+1} \mid x_{\le t})$ and three notes: shifted targets, the target at position t being the input at $t+1$; no peeking, the causal mask hiding the answer from each position; and scale, Llama 3 being trained on more than 15 trillion tokens. The values are illustrative.

Details Training a language model needs no labels beyond the text itself: the target at each position is simply the next token. Given a sequence $x_1, \ldots, x_{T+1}$, the model is run on $x_1, \ldots, x_T$ with the causal mask, producing at each position t a distribution over what x_{t+1} should be, and the loss at that position is the negative log of the probability it assigned to the token that actually came next. The slide's numbers: at position 1 the model gave *cat* probability .055 after *The*, a loss of $-\ln .055 = 2.90$; at position 4 it gave *the* probability .67 after *on*, a loss of 0.40; at position 5 the running example's .507 for *mat*, 0.68. The average over positions, 1.72, is the training loss for this sequence, and its exp, 5.6, is the perplexity.

Three things make this efficient. All T predictions come from one forward pass, because the mask lets every position see exactly its own prefix (slide 14); without the mask the model would have to be run T times on growing prefixes. All T losses are averaged and one backward pass computes the gradient of the average with respect to every parameter; a single sequence of 4,096 tokens therefore supplies 4,096 training signals. And the targets are free: any text is training data, which is why the corpus can be trillions of tokens. The loss is cross-entropy between the one-hot target and the predicted distribution, and its gradient at the logits has a simple form, $p - e_{\text{target}}$: push the correct token's logit up and every other logit down in proportion to the probability it was wrongly given.

The optimisation is stochastic gradient descent in the Adam family (AdamW in practice): a batch of sequences, often several million tokens in total, gives one gradient estimate, the parameters are moved a small step against it, and the process repeats for hundreds of thousands of steps. The learning rate is warmed up over the first steps and decayed toward the end (slide 19 explains why pre-norm makes the warm-up short). The numbers are large: GPT-2 was trained on about 40 GB of text; GPT-3 on about 300 billion tokens; Llama 3 on 15.6 trillion. How to choose the number of tokens relative to the number of parameters is the subject of the scaling-laws lecture; the rule of thumb from Hoffmann et al. (2022) is about twenty tokens per parameter for a compute-optimal model, and models meant for cheap inference are trained far beyond that, as Llama 3 8B's 15.6T tokens (nearly 2,000 per parameter) show.

The same loss, computed on held-out text, is the model's basic quality measure, and it is the quantity whose smooth decrease with model size, data and compute the scaling laws describe. Everything downstream, from instruction following to reasoning, is first a matter of lowering this number.

$\mathscr{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_{t+1} \mid x_1, \ldots, x_t)$; perplexity $= e^{\mathscr{L}}$

gradient at the logits of one position: $\partial \mathscr{L}_t / \partial z = p - e_{x_{t+1}}$

slide: losses $(2.90, 3.41, 1.20, 0.40, 0.68)$, mean 1.72, perplexity 5.6

Terms

cross-entropy loss $-\log p(\text{correct token})$ at each position, averaged over positions; the training objective of language models.

self-supervised (next-token) training Training on unlabelled text by using each next token as the target for the position before it.

target shift The alignment in which the target for input position t is the token at position $t+1$.

gradient descent Adjusting every parameter a small step in the direction that lowers the loss, using gradients computed by backpropagation.

backpropagation Computing the gradient of the loss with respect to every parameter by applying the chain rule backwards through the network.

AdamW The optimiser used for nearly all LLM training: gradient descent with per-parameter adaptive step sizes, momentum, and decoupled weight decay (Loshchilov and Hutter 2017).

batch The set of sequences whose gradients are averaged for one parameter update; in LLM training, millions of tokens.

training tokens D The total number of tokens the model is trained on; 15.6 trillion for Llama 3.

Intuition

Training is a reading exercise with a quiz after every word: the model guesses the next word, is told the answer, and adjusts. The trick that makes it affordable is that all the quizzes for a whole passage are graded at once, because the mask stops each guess from seeing its answer. A perplexity of 5.6 on the slide means the model was, on average, as unsure as if it had been choosing among 5.6 equally likely words; a good model on ordinary English scores well under 10 on that scale, and the whole scaling story is about pushing it down.

10.2 Slide 27: Where the parameters live: GPT-2 small

124,439,808 weights, counted from the architecture. The MLPs hold the largest share.

What the slide shows A horizontal stacked bar of the model's parameters by component: token embeddings 31.0% (green), position table 0.6% (violet), attention 22.8% (orange), MLP 45.5% (blue), norms 0.03% (amber). Below, the table behind it, with the formula for each component, the instantiation for GPT-2 small, and the count: token embedding E, $Vd = 50{,}257 \times 768 = 38{,}597{,}376$; position table, $T_{\max}d = 1{,}024 \times 768 = 786{,}432$; attention, $4d^2 + 4d$ per block times 12 blocks, 28,348,416; MLP, $8d^2 + 5d$ per block times 12, 56,669,184; norm gains and biases, $2d$ per norm times 25 norms, 38,400; unembedding, tied to E, 0; total 124,439,808. Two notes: in a small model the vocabulary table alone is almost a third of the weights; in a large one the blocks dominate, since $12Ld^2$ with $L = 32$, $d = 4096$ gives 6.4B and Llama-2 7B has 6.7B in total.

Details The slide is the deck's one exact count, and every line can be reproduced from the configuration $V = 50{,}257$, $d = 768$, $T_{\max} = 1{,}024$, $L = 12$, $h = 12$, $d_{\text{ff}} = 3{,}072$.

- **Token embedding**: one row per vocabulary entry, $50{,}257 \times 768 = 38{,}597{,}376$.
- **Position table**: GPT-2 uses a learned table (slide 8), one row per position up to its context of 1,024: $1{,}024 \times 768 = 786{,}432$.
- **Attention, per block**: four $d \times d$ matrices (W_Q, W_K, W_V, W_O) with a bias each: $4d^2 + 4d = 2{,}359{,}296 + 3{,}072 = 2{,}362{,}368$; times 12 blocks, 28,348,416.
- **MLP, per block**: W_1 is $4d \times d$ with a bias of $4d$, W_2 is $d \times 4d$ with a bias of d: $8d^2 + 5d = 4{,}718{,}592 + 3{,}840 = 4{,}722{,}432$; times 12, 56,669,184.
- **Norms**: LayerNorm has a gain and a bias, $2d = 1{,}536$; two per block and one final make 25: 38,400.
- **Unembedding**: tied to E (slide 23), so no new parameters.

The sum is 124,439,808, which is the number reported by the public implementations. The slide's percentages follow: $38.6/124.4 = 31.0\%$, $0.79/124.4 = 0.6\%$, $28.3/124.4 = 22.8\%$, $56.7/124.4 = 45.5\%$, $0.038/124.4 = 0.03\%$. Within the blocks the MLP-to-attention ratio is 2.0, the $8d^2$ to $4d^2$ of slide 21.

The lesson is in the second note. The vocabulary table scales as Vd and the blocks as $12Ld^2$, so the table's share is roughly $V/(12Ld)$: for GPT-2 small, $50{,}257/(12 \times 12 \times 768) = 0.45$ relative to the blocks, hence a third of the whole. For Llama 2 7B, $32{,}000/(12 \times 32 \times 4{,}096) = 0.02$, and the blocks are 96% of the model even with an untied head. In large models the parameter count is the block count, and the block count is $12Ld^2$ to within a few percent; the remaining differences come from the SwiGLU width (Llama 7B's 11,008 rather than $4d = 16{,}384$ makes its MLP $3 \times 4{,}096 \times 11{,}008 = 135$M per layer against $8d^2 = 134$M) and from grouped-query attention shrinking W_K and W_V.

Counting parameters is also the first step in costing a model: at two bytes per parameter in 16-bit formats, GPT-2 small is 249 MB and Llama 2 7B is 13.5 GB of weights, which must be read from memory for every generated token (slide 28).

GPT-2 small: $Vd + T_{\max}d + L(4d^2 + 4d) + L(8d^2 + 5d) + (2L+1)\,2d = 124{,}439{,}808$

table share of the whole $\approx \dfrac{Vd}{Vd + 12Ld^2}$: GPT-2 small 0.31; Llama-2 7B 0.02 (if tied) or 0.04 (untied, as built)

Terms

parameter count The total number of learned numbers in a model, the "124M" or "7B" in its name; computed from the configuration as on this slide.

configuration The handful of numbers that define a transformer's shape: V, d, L, h, d_{ff}, $T_{\max}$, and whether the head is tied.

position table parameters $T_{\max}d$ for a learned absolute table (GPT-2); zero for sinusoidal or rotary encodings.

tied parameters Parameters shared between two uses and counted once, such as the embedding and unembedding of GPT-2.

non-embedding parameters The parameters in the blocks, excluding the token and position tables; the count the scaling laws use, because it is what sets per-token compute.

Intuition

A model's parameters are a ledger with three big accounts: the dictionary (embedding), the readers (attention) and the memory (MLP). In a small model the dictionary dominates because it is priced per word, not per layer; in a large one the readers and the memory, priced as d^2 per layer, take over. Being able to reproduce this slide's table from a model card is the test of whether the architecture has been understood.

10.3 Slide 28: Compute cost: linear in the weights, quadratic in the context

Per layer, the matrix products scale with T and the attention pattern with T^2. The two cross at $T = 6d$.

What the slide shows A log–log plot of multiply–adds per layer against context length T from 100 to a million tokens, for $d = 4{,}096$. A straight ink line, $12Td^2$, the weight products, rises with slope 1; an orange line, $2T^2d$, the scores and mixing, rises with slope 2 and crosses it at $T = 6d \approx 25$k, marked by a dashed vertical. Four notes: $12Td^2$ is the Q, K, V, O projections and the MLP, linear in T; $2T^2d$ is every token against every earlier token, quadratic; memory is the $T \times T$ attention matrix, which FlashAttention avoids storing by computing it in tiles, and the KV cache; and decode touches every weight once per token, so single-stream generation is usually limited by memory bandwidth.

Details Per layer and per forward pass over T tokens, the arithmetic has two parts with different growth. The weight products, $12Td^2$ multiply–adds, come from passing each of the T positions through the block's $12d^2$ weights: $4d^2$ in the attention projections and $8d^2$ in the MLP. This is the "2 FLOPs per parameter per token" rule, and it is linear in T. The attention pattern, $2T^2d$, comes from the $T \times T$ score matrix: $QK^\top$ costs T^2d_k per head and AV another T^2d_k, and summing over h heads of width d/h gives $2T^2d$. It is quadratic in T and has no weights in it at all. The two are equal when $12Td^2 = 2T^2d$, that is at $T = 6d$: 24,576 tokens for $d = 4{,}096$. Below that, the model is dominated by its weights and attention is a minor cost; above it, attention dominates and each doubling of context quadruples its cost. The plot makes the crossing visible, and also shows why the quadratic term was ignorable for years: at the 1,024- or 2,048-token contexts of GPT-2 and GPT-3, attention was a few percent of the arithmetic.

The memory story is different and was the practical problem first. The score matrix is $T \times T$ per head; materialising it for $T = 8{,}192$ and 32 heads means two billion numbers per layer, and in training the matrices of every layer are kept for the backward pass. FlashAttention (Dao et al. 2022) computes exactly the same result without ever writing the full matrix: it processes blocks of queries against blocks of keys in the GPU's fast on-chip memory, keeping running maxima and sums so that the softmax can be normalised at the end, and recomputes the pattern in the backward pass rather than storing it. The arithmetic is unchanged, the memory traffic falls by an order of magnitude, and the result is both less memory and more speed, because attention at these sizes was bound by memory reads, not arithmetic.

The decode note connects to slide 25. During single-sequence generation each new token passes through every weight once, $2N$ FLOPs for N parameters, and reading those N parameters from memory takes $2N$ bytes at 16 bits: one FLOP per byte, against the several hundred FLOPs per byte an accelerator can sustain. The step time is set by bandwidth (weights plus KV cache, divided by bytes per second), and the arithmetic units sit mostly idle. Batching many sequences together shares each weight read across them and recovers efficiency, which is why serving systems batch aggressively and why the cache size, which limits the batch, matters so much. Training, by contrast, processes thousands of tokens per weight read and is compute-bound; its total cost is the familiar $6ND$ FLOPs for N parameters and D tokens (2 forward, 4 backward per parameter per token), 3.8×10^{25} for Llama 3 405B at 15.6 trillion tokens.

per layer, T tokens: weights $12Td^2$, attention $2T^2d$ multiply–adds; equal at $T = 6d$

training FLOPs $\approx 6ND$; decode FLOPs per token $\approx 2N$ against $2N$ bytes of 16-bit weights: 1 FLOP per byte

$d = 4{,}096$: crossover $T = 24{,}576$; at $T = 2{,}048$ attention is $2T^2d/(12Td^2) = T/6d \approx 8\%$ of the weight cost

Terms

multiply–add One multiplication and one addition, the unit of work in a matrix product; counted as two FLOPs.

quadratic attention cost The T^2 growth of the score and mixing computation with context length, because every position is compared with every earlier one.

crossover $T = 6d$ The context length at which the attention term equals the weight term per layer.

tiling Computing a large matrix operation in blocks that fit in fast on-chip memory; FlashAttention's method of avoiding the full score matrix.

arithmetic intensity FLOPs performed per byte read from memory; decode at batch 1 has about 1 FLOP per byte, far below what accelerators need to be busy.

memory-bandwidth bound Limited by how fast data can be read from memory rather than by arithmetic; single-sequence decode is.

$6ND$ rule The training cost estimate of 6 FLOPs per parameter per token: 2 in the forward pass and 4 in the backward pass.

Intuition

Two separate bills: one for the weights, charged per token, and one for attention, charged per pair of tokens. For short contexts the weight bill dominates and the model's cost is its size; for long contexts the pair bill takes over and the cost is the context. FlashAttention did not reduce the pair bill, it stopped paying a third bill, for memory, that was larger than both. And at generation time the real constraint

> is often none of these but the time to fetch the weights from memory, which is why "how fast does it generate?" is answered in gigabytes per second, not FLOPs.

11 The family and the whole machine (slides 29–31)

11.1 Slide 29: The family, and what changed since 2017

Three ways to wire the same block, and the refinements that separate current LLMs from the original design.

What the slide shows On the left, three icons showing which positions each token may read: a full square (every token sees both directions) for *encoder-only*, BERT, used for embeddings and classification; a full square beside a masked triangle for *encoder–decoder*, the 2017 Transformer and T5, in which the decoder also attends to the encoder, used for translation; and a masked triangle alone for *decoder-only*, GPT, Llama, Mistral, Qwen, Gemma, with causal attention, "this deck". On the right, a then-and-now table: layout, encoder–decoder then and decoder-only now; norm, LayerNorm after then and RMSNorm before now; position, sinusoids added then and RoPE in attention now; activation, ReLU then and SwiGLU or GELU now; attention, multi-head then and grouped-query now. Two notes: grouped-query attention has several query heads share one key/value head, shrinking the KV cache, with Llama-3 70B at 64 query heads and 8 key/value heads; and the residual stream, dot-product attention and the MLP are unchanged.

Details The block of slide 21 can be wired into a model in three ways, distinguished by the mask and by whether there is a second stream to attend to.

Encoder-only models (BERT, 2018) use no causal mask: every position reads every other, so each output vector summarises the whole input in both directions. That is the right design for understanding a complete text, for example classifying it or producing a vector for retrieval, but it cannot generate, because there is no position whose view is restricted to a prefix. *Encoder–decoder* models (the 2017 Transformer, T5) have both: an encoder reads the input bidirectionally, and a decoder generates the output with a causal mask, with an extra *cross-attention* sublayer in each decoder block whose queries come from the decoder and whose keys and values come from the encoder's output. It is the natural design for translation and other input-to-output tasks. *Decoder-only* models drop the encoder and treat everything, prompt and answer, as one sequence under a causal mask. The prompt plays the role the encoder's input played, read through ordinary causal self-attention, which turns out to be sufficient, and the single stream makes the model simpler to train and to scale. Since GPT-2 and GPT-3 showed that this design with enough data does everything the others do, it has been the design of essentially every LLM.

The table lists what changed inside the block over seven years; each row is a slide of this deck. Pre-norm replaced post-norm for stability at depth (slide 19), and RMSNorm replaced LayerNorm because centering turned out to be dispensable (slide 20). RoPE replaced additive sinusoids because relative position is what attention needs and injecting it at the scores extends better (slide 9). SwiGLU replaced ReLU for a small, consistent gain in loss at equal parameters (slide 18). Grouped-query attention (Ainslie et al. 2023) replaced multi-head attention in the key/value projections: the h query heads are divided into groups, each group shares one key head and one value head, so W_K and W_V shrink from $d \times d$ to $d \times h_{kv} d_k$ and, more importantly, the KV cache (slide 25) shrinks by the factor h/h_{kv}. Llama 3 70B uses 64 query heads and 8 key/value heads, an eightfold cache reduction for a small quality cost; the limiting case of a single shared key/value head is multi-query attention (Shazeer 2019). Other common changes not in the table: biases are dropped from all linear layers, and vocabularies grew (slide 5).

The last note is the one to remember. Everything in the table is a refinement, worth a few percent of loss or a large factor of memory. The residual stream, dot-product attention with the causal mask, and the

position-wise MLP are the same as in 2017, and a reader of the original paper would recognise every current model.

Terms

encoder-only A transformer with bidirectional (unmasked) attention, producing a contextual vector for each input position; BERT. Suited to classification and embeddings, not generation.

encoder–decoder The 2017 design: a bidirectional encoder over the input and a causal decoder over the output, linked by cross-attention; T5. Suited to translation.

cross-attention Attention whose queries come from one sequence (the decoder) and whose keys and values come from another (the encoder's output).

grouped-query attention (GQA) Attention in which groups of query heads share a key/value head, so $h_{kv} < h$; shrinks W_K, W_V and the KV cache by h/h_{kv} (Ainslie et al. 2023).

multi-query attention (MQA) The extreme of GQA with a single key/value head shared by all query heads (Shazeer 2019).

BERT A 2018 encoder-only model trained to fill in masked tokens; the standard example of the bidirectional design.

T5 A 2019 encoder–decoder model that cast every task as text-to-text; the standard example of that design.

Intuition

Three wirings, one question: who may read whom? Everyone reads everyone (encoder); a writer reads only its own past plus a finished input (encoder–decoder); a writer reads only its own past (decoder-only). The third won because a long enough past can contain the input. The then-and-now table is a list of knobs turned by a notch; the picture of the machine is the same, which is why a 2017 paper is still the right place to start.

11.2 Slide 30: The whole machine on one slide

Seven components, one loop: tokens in, a distribution out, sample, append, repeat.

What the slide shows The map of slide 3 redrawn with seven numbered badges on its boxes, and a legend: 1 Tokenizer, text to subword IDs; 2 Embedding, each ID to a vector of width d; 3 Position, order added at the input or rotated into q, k; 4 Attention, each token reads earlier tokens with Q, K, V and h heads; 5 MLP, a per-token transform holding most of the weights; 6 Norm and residual, stable scale, with every sublayer adding to the stream; 7 Unembed and softmax, the final vector to a distribution over V tokens.

Details The slide is the summary, and the notes use it to collect the deck's numbers in one place for GPT-2 small ($V = 50{,}257$, $d = 768$, $L = 12$, $h = 12$, $T_{\max} = 1{,}024$) and, where they differ in kind, for a Llama-style model.

1. **Tokenizer** (slide 5): byte-level BPE, $V = 50{,}257$; about four characters per token in English. No parameters in the network.
2. **Embedding** (slides 6–7): $E \in \mathbb{R}^{V \times d}$, 38.6M parameters, 31% of the model; a static, non-contextual vector per token.
3. **Position** (slides 8–9): GPT-2 adds a learned table (0.8M parameters); Llama rotates q and k with RoPE (no parameters). Without it the model is permutation-invariant.

4. **Attention** (slides 10–16): per block W_Q, W_K, W_V, W_O, $4d^2 = 2.4$M parameters; 12 heads of width 64; causal mask; cost $2T^2d$ per layer in the pattern. The only place positions interact.
5. **MLP** (slides 17–18): per block $768 \to 3{,}072 \to 768$, $8d^2 = 4.7$M parameters, two thirds of the block; GELU in GPT-2, SwiGLU in Llama. The only nonlinearity applied to the stream's content.
6. **Norm and residual** (slides 19–22): pre-norm, LayerNorm ($2d$ parameters each, 25 in all) in GPT-2, RMSNorm in Llama; $x \leftarrow x + f(\text{Norm}(x))$ twice per block; 12 blocks of about $12d^2 = 7.1$M, giving 85M in the stack.
7. **Unembed and softmax** (slides 23–24): $W_U = E^\top$ (tied) in GPT-2, separate in Llama; V logits, softmax, then a sampling rule.

The loop around it (slide 2) is generation; the loss on its outputs (slide 26) is training; the keys and values it computes along the way are the cache (slide 25). Total parameters 124,439,808 (slide 27); cost per token per layer $12d^2+$ attention (slide 28).

A reader who can, for any model card, fill in these seven lines (the vocabulary and tokenizer, the width, the position scheme, the number of heads and key/value heads, the MLP width and activation, the norm type and placement, and whether the head is tied) has the whole architecture, and the rest of the card is training details.

Terms

model card The public description of a released model: its configuration, training data and evaluation; the source of every dimension quoted in the deck.

architecture The fixed structure of the network, the seven components and their shapes, as opposed to the learned weights that fill it.

forward pass One application of the whole model to a sequence, producing a distribution at each position.

Intuition

Seven parts, and only two of them think: attention (reading) and the MLP (recalling). The other five are a dictionary at each end, a way of counting positions, and the plumbing that keeps the numbers tame. If the deck is remembered as a single image, it should be slide 3's map with slide 4's lanes drawn through it: vectors flowing left to right, crossing only at attention, each ending in a guess about the next word.

11.3 Slide 31: Sources and further reading

Cited dimensions and identifiers were checked against papers and model cards. Illustrative values are invented.

What the slide shows Two columns of references in small type, grouped by theme. Architecture: Vaswani et al. 2017 (Attention Is All You Need), He et al. 2016 (residual networks). Tokens, embeddings, output: Sennrich, Haddow and Birch 2016 (BPE for translation), Mikolov et al. 2013 (word2vec), Press and Wolf 2017 (weight tying). Position, normalization, activation: Su et al. 2021 (RoPE), Ba, Kiros and Hinton 2016 (LayerNorm), Zhang and Sennrich 2019 (RMSNorm), Xiong et al. 2020 (pre-norm), Hendrycks and Gimpel 2016 (GELU), Shazeer 2020 (GLU variants). What the parts learn: Elhage et al. 2021 (transformer circuits), Clark et al. 2019 (what BERT looks at), Tenney, Das and Pavlick 2019 (the classical pipeline), Geva et al. 2021 (feed-forward layers as key–value memories), Olsson et al. 2022 (induction heads). Decoding and efficiency: Holtzman et al. 2020 (nucleus sampling), Dao et al. 2022 (FlashAttention), Ainslie et al. 2023

(GQA). Models cited for dimensions: Radford et al. 2019 (GPT-2, with its public configuration and code), Brown et al. 2020 (GPT-3), Touvron et al. 2023 (Llama 2), Dubey et al. 2024 (Llama 3).

Details The slide separates two kinds of number in the deck. *Checked* numbers are dimensions and identifiers of real models: GPT-2 small's 12/768/12/50,257/1,024, the parameter counts of slide 22, Llama 3 70B's 64 and 8 heads, Llama 3's 15 trillion training tokens. These come from the papers listed and from the public configuration files that accompany the released weights, and the notes have reproduced the arithmetic where it could be done by hand. *Illustrative* numbers are the token IDs on slide 5, the attention scores and weights on slides 10–16, the logits and probabilities on slides 2, 23, 24 and 26, and the toy vectors on slide 20; they were invented to make a mechanism visible, chosen to be internally consistent (the softmax on slide 13 really does produce the weights on slides 10 and 11; the LayerNorm on slide 20 really does produce the bars shown), and should not be quoted as measurements.

The reading list in Appendix D expands the slide's references into one list per section of the deck, with a line on why each paper is worth reading, and is matched by the paper archive distributed with these notes. For a first pass, five papers cover the deck: Vaswani et al. (2017) for the architecture, Radford et al. (2019) for the decoder-only recipe as it is used today, Su et al. (2021) for RoPE, Dao et al. (2022) for why attention is implemented the way it is, and Dubey et al. (2024) for what a current model's configuration and training look like at scale. For the interpretability thread, Elhage et al. (2021) is the place to start.

Terms

checked value A number taken from a paper, model card or public configuration file and verified against it.

illustrative value A number invented for a figure to show a mechanism; internally consistent but not a measurement.

configuration file The machine-readable description of a released model's shape (layers, width, heads, vocabulary) distributed with its weights; the primary source for architectural dimensions.

Intuition

Every number in a lecture is either measured or made up, and the honest thing is to say which. The deck's made-up numbers are made up carefully, so that a student who recomputes them gets the same answer; its measured numbers are the ones to carry into other contexts. When in doubt about a figure in any slide deck, the question is: which paper, which table, and could I recompute it?

A Notation

symbol	meaning	notes
T	sequence (context) length	tokens in the input; also the sampling temperature on slide 24, by context
$T_{\max}$	maximum context length	1,024 for GPT-2, 8,192 for Llama 3; the size of a learned position table
V	vocabulary size	50,257 GPT-2; 32,000 Llama 2; 128,256 Llama 3; 256,000 Gemma
d	model width	length of a token's vector; d_{model} in papers
L	depth (number of blocks)	also the loss $\mathscr{L}$, written with a script letter
h, h_{kv}	query heads, key/value heads	$h_{kv} = h$ for multi-head attention, smaller for GQA
d_k	head width	d/h; 64 in GPT-2 small, 128 in GPT-3 and Llama
d_{ff}	MLP hidden width	$4d$ in GPT-2; $\frac{8}{3}d$ to $3.5d$ with SwiGLU
x_t, X	token vector, stream matrix	$x_t \in \mathbb{R}^d$ column vector; $X \in \mathbb{R}^{T\times d}$ with one row per token
E	embedding table	$V \times d$; row i is token i's input vector
PE_t, $P[t]$	position vector	sinusoidal formula or learned table row; added to x_t
θ_i, ω_i	RoPE and sinusoid frequencies	both $10000^{-2i/d}$; θ_i with $d = d_k$
$R(\alpha)$	rotation	block-diagonal, pair i turned by $\alpha\theta_i$
W_Q, W_K, W_V, W_O	attention projections	each $d \times d$ overall ($d_k \times d$ per head); $4d^2$ per block
Q, K, V	queries, keys, values	$T \times d_k$ per head; $Z = AV$ the head output
s_{ij}, α_{ij}	score, attention weight	$s_{ij} = q_i \cdot k_j/\sqrt{d_k}$; α = row-softmax of s after the mask
A, M	attention matrix, mask	$T \times T$; $M_{ts} = 0$ for $s \le t$, $-\infty$ otherwise
W_1, W_2, W_3	MLP matrices	W_1 up, W_2 down, W_3 the SwiGLU signal path
σ	activation function	ReLU, GELU or SiLU; also a standard deviation in the norm, by context
μ, RMS	mean, root mean square	over the d coordinates of one token
γ, β	norm gain, norm bias	length d; RMSNorm has no β
W_U	unembedding	$V \times d$; $E^\top$ when tied
z, p	logits, probabilities	$p = \text{softmax}(z)$, length V
k, p (sampling)	top-k count, nucleus mass	slide 24
N, D	parameters, training tokens	in the $6ND$ training-cost rule

B Formulas at a glance

Input. $x_t = E[\mathrm{id}_t]$ (+ PE_t or $P[t]$ for additive position schemes); $X \in \mathbb{R}^{T\times d}$.

Sinusoidal position. $PE_{t,2i} = \sin(t\omega_i)$, $PE_{t,2i+1} = \cos(t\omega_i)$, $\omega_i = 10000^{-2i/d}$; PE_{t+k} is a fixed linear function of PE_t.

RoPE. $\tilde{q}_m = R(m)q$, $\tilde{k}_n = R(n)k$ with pair i rotated by $m\theta_i$, $\theta_i = 10000^{-2i/d_k}$; $\langle \tilde{q}_m, \tilde{k}_n\rangle = \langle R(m-n)q, k\rangle$.

Attention (one head). $Q = XW_Q^\top$, $K = XW_K^\top$, $V = XW_V^\top$; $A = \mathrm{softmax}_{\mathrm{rows}}(QK^\top/\sqrt{d_k} + M)$; $Z = AV$.

Causal mask. $M_{ts} = 0$ if $s \le t$, $-\infty$ otherwise; after the softmax $\alpha_{ts} = 0$ for $s > t$.

Multi-head. $\mathrm{MHA}(X) = [Z^{(1)}|\cdots|Z^{(h)}]\,W_O^\top$, $d_k = d/h$; weights $4d^2$ regardless of h. GQA: $h_{kv} < h$ key/value heads, W_K, W_V of size $h_{kv}d_k \times d$.

MLP. $W_2\,\sigma(W_1x + b_1) + b_2$, $d \to d_{\mathrm{ff}} \to d$; weights $2dd_{\mathrm{ff}} = 8d^2$ at $d_{\mathrm{ff}} = 4d$. SwiGLU: $W_2(\mathrm{SiLU}(W_1x) \odot W_3x)$, $3dd_{\mathrm{ff}} = 8d^2$ at $d_{\mathrm{ff}} = \frac{8}{3}d$.

Activations. ReLU $\max(0,x)$; GELU $x\Phi(x) \approx 0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$; SiLU $x/(1+e^{-x})$.

Norms. LayerNorm $\gamma \odot (x-\mu)/\sqrt{\sigma^2+\varepsilon} + \beta$; RMSNorm $\gamma \odot x/\sqrt{\frac{1}{d}\sum_c x_c^2 + \varepsilon}$; statistics over one token's d coordinates.

Block (pre-norm). $H = X + \mathrm{MHA}(\mathrm{Norm}_1(X))$; $X' = H + \mathrm{MLP}(\mathrm{Norm}_2(H))$. Post-norm (2017): $\mathrm{Norm}(X + F(X))$.

Output. $z = W_U\,\mathrm{Norm}(h_T)$; $p_i = e^{z_i}/\sum_j e^{z_j}$; tied head $W_U = E^\top$.

Sampling. temperature $p_i \propto e^{z_i/T}$; top-k: keep k largest, renormalise; top-p: smallest set with cumulative mass $\ge p$, renormalise.

Loss. $\mathscr{L} = -\frac{1}{T}\sum_t \log p_\theta(x_{t+1} \mid x_{\le t})$; perplexity $e^{\mathscr{L}}$; gradient at the logits $p - e_{\mathrm{target}}$.

Parameters. per block $4d^2 + 8d^2 = 12d^2$ (+ biases, + $2d$ per norm); stack $12Ld^2$; embedding Vd (+ Vd untied head, + $T_{\max}d$ learned positions). GPT-2 small: 124,439,808.

Compute per layer, T tokens. weights $12Td^2$, attention $2T^2d$ multiply–adds; equal at $T = 6d$. Training $\approx 6ND$ FLOPs; decode $\approx 2N$ FLOPs per token.

KV cache. $2LT\,h_{kv}d_k$ numbers ($= 2LTd$ for MHA); bytes $= \times 2$ at 16 bits. Decode cost per layer per step with cache: $12d^2 + 2td$.

Worked numbers (derived). GPT-2 small: E 38.6M (31.0%), attention 28.3M (22.8%), MLP 56.7M (45.5%). Llama-2 7B shape at $T = 4{,}096$, 16-bit: cache 2^{31} B $= 2$ GiB. Llama 3 405B: $6 \times 405\text{B} \times 15.6\text{T} \approx 3.8 \times 10^{25}$ FLOPs.

C Exercises

1. **Tokens and cost.** A 2,000-word English document is about 10,000 characters. Using the slide-5 rule of thumb, how many tokens is it? If a second tokenizer produces 20% fewer tokens for the same text, by what factor does the attention term $2T^2d$ fall, and the weight term $12Td^2$? Which of the two matters at $d = 4{,}096$ and this length (compare with $T = 6d$)?

2. **Embedding share.** Llama 3 8B has $V = 128{,}256$, $d = 4{,}096$, $L = 32$ and an untied head. Compute the embedding and unembedding parameters, the $12Ld^2$ block estimate, and the tables' share of the total. Repeat for GPT-2 small ($V = 50{,}257$, $d = 768$, $L = 12$, tied) and explain the difference in one sentence.

3. **Sinusoids.** For $d = 512$, give the period in positions of pairs $i = 0$, $i = 128$ and $i = 255$. Show that for a single pair the map from $(PE_{t,2i}, PE_{t,2i+1})$ to $(PE_{t+k,2i}, PE_{t+k,2i+1})$ is a rotation by $k\omega_i$, so that PE_{t+k} is a linear function of PE_t with coefficients that do not depend on t.

4. **RoPE range.** With $d_k = 128$ and base 10,000, pair i turns by θ_i per position. Find the smallest i whose pair completes less than one full turn over 8,192 positions. Repeat with base 500,000 (Llama 3). What fraction of the pairs, in each case, never repeat an angle within the training context?

5. **Softmax by hand.** In slide 13's row for *she*, change the score on *Bob* from 1.4 to 3.0 and recompute the five weights. Then suppose these scores were unscaled dot products with $d_k = 64$: divide by $\sqrt{64}$ and recompute. Comment on what the scaling does to the sharpness of the row.

6. **Mask arithmetic.** How many entries of a $T \times T$ causal attention matrix are non-zero? For $T = 4{,}096$, what fraction of the full T^2 score computation does a causal model actually need, and what does that say about the $2T^2d$ estimate of slide 28?

7. **Heads are free.** Show that the attention parameters of a block are $4d^2$ for any number of heads h dividing d, and that the score and mixing cost is $2T^2d$ for any h. Then compute, for GPT-3 ($d = 12{,}288$, $h = 96$) at $T = 2{,}048$, the attention parameters per layer and the multiply–adds per layer in the pattern.

8. **SwiGLU width.** Llama's rule for the SwiGLU hidden width is $\frac{8}{3}d$ rounded up to a multiple of 256. Compute it for $d = 4{,}096$ and compare with the 11,008 in Llama 7B's configuration. Give the MLP parameter count $3dd_{\text{ff}}$ and compare with $8d^2$. Repeat for Llama 3 8B's $d_{\text{ff}} = 14{,}336$: what multiple of d is it, and how far above $8d^2$ is the MLP?

9. **Norms by hand.** For $x = (1,2,3,4,5,6,7,8)$ compute μ, σ and RMS, then LayerNorm(x) and RMSNorm(x) with $\gamma = 1$, $\beta = 0$, $\varepsilon = 0$. Verify that LayerNorm's output has mean 0 and RMS 1, and find the mean and RMS of RMSNorm's output.

10. **Why residuals.** Suppose each of L sublayers, without a residual connection, multiplies the gradient by a factor 0.9 on average. What is the factor after $L = 24$ sublayers (12 blocks) and after $L = 252$ (126 blocks)? With residual connections, the Jacobian is $I + J_l$ with $\|J_l\| \approx 0.1$; bound the product's deviation from the identity for the same L and comment.

11. **Count a model.** Llama 2 7B has $V = 32{,}000$, $d = 4{,}096$, $L = 32$, $h = 32$, $d_{\text{ff}} = 11{,}008$, RMSNorm, no biases, RoPE, and an untied head. Count the parameters component by component (embedding, attention per layer, MLP per layer, norms, head) and compare with the published 6.74B.

12. **KV cache at scale.** Llama 3 70B has 80 layers, 8 key/value heads of width 128. Compute the cache in bytes per token at 16 bits, and the total for a 128k-token context. What would it be with 64 key/value

heads (plain multi-head)? With the 16-bit weights (140 GB) spread over four 80 GB GPUs, how many 128k-token sequences can the remaining memory hold, and how many could it hold with the multi-head cache?

13. **Crossover and training cost.** For Llama 3 405B ($d = 16{,}384$), find $T = 6d$. At a 128k context, which term dominates per layer, and by what factor? Compute $6ND$ for $N = 405\text{B}$ and $D = 15.6\text{T}$ tokens, and the wall-clock time on 16,000 H100s each sustaining 400 TFLOP/s.

14. **Decode speed.** A 70B model in 16-bit weights is 140 GB. On a GPU with 3.35 TB/s of memory bandwidth, what is the minimum time per generated token at batch 1, ignoring the cache, and the resulting tokens per second? Repeat with 4-bit weights. How many FLOPs does one token need ($2N$), and what fraction of a 989 TFLOP/s peak is used at batch 1?

15. **Sampling arithmetic.** Starting from slide 23's probabilities $(.507, .233, .133, .066, .056, .005)$, compute the distribution after top-k with $k = 3$; after top-p with $p = 0.8$; and at temperature $T = 2$ (work from the logits $(5.10, 4.32, 3.76, 3.06, 2.90, 0.50)$). Which setting changes the probability of *mat* the most?

16. **Loss and perplexity.** Slide 26's losses are $(2.90, 3.41, 1.20, 0.40, 0.68)$. Compute the perplexity of the mean loss. A model with an average loss of 2.0 nats per token: what is its perplexity, and its loss in bits per token? If a tokenizer change makes each token carry 20% more characters, why is a comparison of per-token loss across the two tokenizers meaningless, and what should be compared instead?

D Reading list by section

The papers behind each part of the deck, in the order they are discussed, with one line on why each is worth reading. Entries with an arXiv identifier are included in the paper archive, the zip file that accompanies these notes; entries without one are listed with a URL. Items marked *secondary source* are blog posts, code repositories or web articles.

1. Overview and the residual stream (slides 1-4)

- **Vaswani et al. (2017).** *Attention is all you need*. arXiv:1706.03762. The original paper: scaled dot-product attention, multi-head attention, sinusoidal positions, the post-norm block and the encoder-decoder layout. Short, and still the right place to start.
- **Radford, Narasimhan, Salimans and Sutskever (2018).** *Improving language understanding by generative pre-training (GPT)*. `https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf`. The first decoder-only transformer language model trained on next-token prediction and then fine-tuned; the recipe the deck describes. Not on arXiv.
- **Radford et al. (2019).** *Language models are unsupervised multitask learners (GPT-2)*. `https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf`. GPT-2: the 124M/355M/774M/1.5B family, pre-norm, byte-level BPE, tied embeddings; the configuration the deck counts by hand. The code and configs are at github.com/openai/gpt-2. Not on arXiv.
- **Elhage et al. (2021).** *A mathematical framework for transformer circuits*. `https://transformer-circuits.pub/2021/framework/index.html`. The residual-stream view of slide 4: heads as independent read-write operations on a shared stream, QK and OV circuits, and the first analysis of induction heads. Published as a web article, not on arXiv.
- **Phuong and Hutter (2022).** *Formal algorithms for transformers*. arXiv:2207.09238. Every component of the deck written out as pseudocode with explicit shapes: tokenization, embeddings, attention, the block, training and decoding. The best companion for checking one's understanding line by line.
- **Bahdanau, Cho and Bengio (2014).** *Neural machine translation by jointly learning to align and translate*. arXiv:1409.0473. Where attention came from: a translation decoder that computes a soft alignment over the source positions instead of compressing the sentence into one vector.
- **Alammar (2018).** *The illustrated transformer (blog)*. `https://jalammar.github.io/illustrated-transformer/`. The most widely read visual walkthrough of the 2017 architecture; the deck's pictures of Q, K, V and multi-head attention follow the same conventions. Secondary source.
- **Karpathy (2022).** *nanoGPT (code repository)*. `https://github.com/karpathy/nanoGPT`. A complete GPT-2 implementation and training loop in about 300 lines of PyTorch; reading model.py after the deck turns every slide into a few lines of code. Secondary source.

2. Tokens, embeddings and position (slides 5-9)

- **Sennrich, Haddow and Birch (2016).** *Neural machine translation of rare words with subword units*. arXiv:1508.07909. Byte-pair encoding as a subword vocabulary for neural models; the method behind every tokenizer on slide 5.

- **Kudo and Richardson (2018).** *SentencePiece: a simple and language independent subword tokenizer and detokenizer for neural text processing*. arXiv:1808.06226. The library used by Llama 2 and many others; treats the input as a raw stream, so whitespace is just another symbol.
- **Kudo (2018).** *Subword regularization: improving neural network translation models with multiple subword candidates*. arXiv:1804.10959. The unigram language-model tokenizer, the main alternative to BPE, and why sampling among segmentations helps training.
- **Tao et al. (2024).** *Scaling laws with vocabulary: larger models deserve larger vocabularies*. arXiv:2407.13623. Why vocabularies grew from 32k to 128k-256k: the compute-optimal vocabulary size rises with model size.
- **Mikolov, Chen, Corrado and Dean (2013).** *Efficient estimation of word representations in vector space*. arXiv:1301.3781. word2vec: word vectors from context prediction, and the king - man + woman = queen regularity of slide 7.
- **Su et al. (2021).** *RoFormer: enhanced transformer with rotary position embedding*. arXiv:2104.09864. RoPE: rotate queries and keys by position so that attention scores depend only on relative distance; the position scheme of Llama, Mistral, Qwen and Gemma (slide 9).
- **Shaw, Uszkoreit and Vaswani (2018).** *Self-attention with relative position representations*. arXiv:1803.02155. The first relative-position scheme for transformers, adding learned offset vectors to keys; the idea RoPE makes parameter-free.
- **Press, Smith and Lewis (2021).** *Train short, test long: attention with linear biases enables input length extrapolation*. arXiv:2108.12409. ALiBi: no position vectors at all, just a distance penalty added to the scores; used by BLOOM and MPT, and the clearest statement of the length-extrapolation problem.
- **Chen, Wong, Chen and Tian (2023).** *Extending context window of large language models via positional interpolation*. arXiv:2306.15595. Rescale RoPE angles so that a longer context maps onto the trained range, then fine-tune briefly; the basis of most context extension.
- **Peng, Quesnelle, Fan and Shippole (2023).** *YaRN: efficient context window extension of large language models*. arXiv:2309.00071. Frequency-aware RoPE rescaling: stretch the slow pairs, leave the fast ones; used by several long-context open models.

3. Attention (slides 10-16)

- **Luong, Pham and Manning (2015).** *Effective approaches to attention-based neural machine translation*. arXiv:1508.04025. Dot-product scoring between a query state and key states, the scoring function the transformer adopted, compared with alternatives.
- **Clark, Khandelwal, Levy and Manning (2019).** *What does BERT look at? An analysis of BERT's attention*. arXiv:1906.04341. Attention heads that track syntactic relations and coreference, and heads that park on [SEP] when idle; the source of slide 16's patterns.
- **Voita, Talbot, Moiseev, Sennrich and Titov (2019).** *Analyzing multi-head self-attention: specialized heads do the heavy lifting, the rest can be pruned*. arXiv:1905.09418. Positional, syntactic and rare-word heads in a translation model, and the finding that most other heads can be removed.
- **Olsson et al. (2022).** *In-context learning and induction heads*. arXiv:2209.11895. Induction heads as the mechanism of in-context learning in small models, and their appearance as a bend in the training curve.

- **Xiao, Tian, Chen, Han and Lewis (2023).** *Efficient streaming language models with attention sinks*. arXiv:2309.17453. The attention sink: idle attention parks on the first token, and keeping it in the cache is what makes sliding-window generation work.
- **Dao, Fu, Ermon, Rudra and Re (2022).** *FlashAttention: fast and memory-efficient exact attention with IO-awareness*. arXiv:2205.14135. Exact attention computed in tiles without materializing the T x T matrix; the implementation every model uses (slide 28).
- **Dao (2023).** *FlashAttention-2: faster attention with better parallelism and work partitioning*. arXiv:2307.08691. The second version, with the work split that reaches a large fraction of peak on H100s.

4. MLP, norm and the block (slides 17-22)

- **Geva, Schuster, Berant and Levy (2021).** *Transformer feed-forward layers are key-value memories*. arXiv:2012.14913. Rows of W1 as pattern detectors and columns of W2 as the updates they write; the reading of the MLP on slide 17.
- **Hendrycks and Gimpel (2016).** *Gaussian error linear units (GELUs)*. arXiv:1606.08415. GELU and SiLU, the smooth activations of BERT, GPT-2 and the SwiGLU gate.
- **Ramachandran, Zoph and Le (2017).** *Searching for activation functions*. arXiv:1710.05941. Swish, x times sigmoid(x), found by search and shown to beat ReLU across tasks; the same function as SiLU.
- **Dauphin, Fan, Auli and Grangier (2017).** *Language modeling with gated convolutional networks*. arXiv:1612.08083. The gated linear unit: one projection gates another elementwise; the structure SwiGLU inherits.
- **Shazeer (2020).** *GLU variants improve transformer*. arXiv:2002.05202. SwiGLU and its relatives as the transformer MLP, with the 8/3 d hidden width that keeps the parameter count fixed (slide 18).
- **He, Zhang, Ren and Sun (2016).** *Deep residual learning for image recognition*. arXiv:1512.03385. Residual connections: learn a correction and add it to the input, which lets very deep networks train (slide 19).
- **Ba, Kiros and Hinton (2016).** *Layer normalization*. arXiv:1607.06450. Normalize over the features of one example rather than over a batch; the norm of the 2017 Transformer, BERT and GPT-2.
- **Zhang and Sennrich (2019).** *Root mean square layer normalization*. arXiv:1910.07467. Drop the centering, keep the scaling: the cheaper norm used by Llama, Mistral, Qwen and Gemma (slide 20).
- **Xiong et al. (2020).** *On layer normalization in the transformer architecture*. arXiv:2002.04745. Why post-norm needs warm-up and pre-norm does not: the gradient scale at initialization under the two placements (slide 19).
- **Tenney, Das and Pavlick (2019).** *BERT rediscovers the classical NLP pipeline*. arXiv:1905.05950. Probing shows syntax resolved in earlier layers and semantics later; the division-of-labor note on slide 22.
- **Tay et al. (2022).** *Scale efficiently: insights from pre-training and fine-tuning transformers*. arXiv:2109.10686. How depth and width trade off at fixed compute; deeper-narrower tends to win downstream.

5. Output and generation (slides 23-25)

- **Press and Wolf (2017).** *Using the output embedding to improve language models*. arXiv:1608.05859. Weight tying: use the embedding matrix as the unembedding; saves V d parameters and improves perplexity (slide 23).
- **Holtzman, Buys, Du, Forbes and Choi (2020).** *The curious case of neural text degeneration*. arXiv:1904.09751. Why greedy and beam search produce bland, repetitive text, and nucleus (top-p) sampling as the fix (slide 24).
- **Fan, Lewis and Dauphin (2018).** *Hierarchical neural story generation*. arXiv:1805.04833. Top-k sampling, introduced here for story generation.
- **Shazeer (2019).** *Fast transformer decoding: one write-head is all you need*. arXiv:1911.02150. Multi-query attention: share one key/value head across all query heads to shrink the KV cache and speed up decoding.
- **Ainslie et al. (2023).** *GQA: training generalized multi-query transformer models from multi-head checkpoints*. arXiv:2305.13245. Grouped-query attention, the middle ground used by Llama 2 70B and all of Llama 3 (slides 25 and 29).
- **Pope et al. (2022).** *Efficiently scaling transformer inference*. arXiv:2211.05102. The cost model of decoding: memory-bound at small batch, the KV cache as the limit, and the partitioning strategies that follow.
- **Kwon et al. (2023).** *Efficient memory management for large language model serving with PagedAttention*. arXiv:2309.06180. vLLM: manage the KV cache in pages like virtual memory, which is what lets a GPU serve many sequences at once.
- **Leviathan, Kalman and Matias (2023).** *Fast inference from transformers via speculative decoding*. arXiv:2211.17192. Draft several tokens with a small model and verify them in one pass of the large one; exact sampling at a fraction of the decode steps.

6. Training and scale (slides 26-28)

- **Loshchilov and Hutter (2017).** *Decoupled weight decay regularization*. arXiv:1711.05101. AdamW, the optimizer used for essentially all LLM training.
- **Kaplan et al. (2020).** *Scaling laws for neural language models*. arXiv:2001.08361. Loss as a power law in parameters, data and compute; the 6ND cost model; and the finding that shape (depth versus width) matters little (slides 22 and 28).
- **Hoffmann et al. (2022).** *Training compute-optimal large language models*. arXiv:2203.15556. Chinchilla: about twenty training tokens per parameter at the compute-optimal point; why current models train on trillions of tokens.
- **Shoeybi et al. (2019).** *Megatron-LM: training multi-billion parameter language models using model parallelism*. arXiv:1909.08053. How the block's matrix products are split across GPUs; the tensor-parallel layout follows the 4 dˆ2 + 8 dˆ2 structure of slide 21.
- **Rajbhandari, Rasley, Ruwase and He (2019).** *ZeRO: memory optimizations toward training trillion parameter models*. arXiv:1910.02054. Where training memory goes (weights, gradients, optimizer states) and how to shard it; the accounting behind training at scale.

- **Brown et al. (2020).** *Language models are few-shot learners*. arXiv:2005.14165. GPT-3: the 96-layer, 12,288-wide, 175B configuration of slide 22, and the demonstration that scale alone yields in-context learning.
- **Touvron et al. (2023).** *Llama 2: open foundation and fine-tuned chat models*. arXiv:2307.09288. The 7B/13B/70B configurations cited on slides 22, 25 and 27: RMSNorm, SwiGLU, RoPE, GQA at 70B.
- **Grattafiori et al. (2024).** *The Llama 3 herd of models*. arXiv:2407.21783. The 8B/70B/405B configurations, 15.6T training tokens, 128k vocabulary, RoPE base 500,000 and GQA with 8 KV heads; the deck's reference for a current model.

7. The family and further reading (slides 29-31)

- **Devlin, Chang, Lee and Toutanova (2018).** *BERT: pre-training of deep bidirectional transformers for language understanding*. arXiv:1810.04805. The encoder-only design of slide 29: bidirectional attention trained to fill in masked tokens.
- **Raffel et al. (2019).** *Exploring the limits of transfer learning with a unified text-to-text transformer*. arXiv:1910.10683. T5, the encoder-decoder design at scale, with a systematic study of the choices that go into one.
- **Touvron et al. (2023).** *LLaMA: open and efficient foundation language models*. arXiv:2302.13971. The first Llama: the decoder-only recipe with pre-RMSNorm, SwiGLU and RoPE that most open models have copied since.
- **Jiang et al. (2023).** *Mistral 7B*. arXiv:2310.06825. A compact current design: GQA, sliding-window attention and a rolling KV cache in a 7B model.
- **Gemma Team (2024).** *Gemma: open models based on Gemini research and technology*. arXiv:2403.08295. A 256k-vocabulary decoder-only model with tied embeddings and GeGLU; a useful contrast with Llama's choices.
- **Qwen Team (2025).** *Qwen2.5 technical report*. arXiv:2412.15115. A current open family with the same block (RMSNorm, SwiGLU, RoPE, GQA) and a 152k vocabulary; what 'typical LLM today' on slide 29 looks like in detail.
- **DeepSeek-AI (2024).** *DeepSeek-V3 technical report*. arXiv:2412.19437. Where the family goes next: multi-head latent attention compressing the KV cache and a mixture-of-experts MLP; the bridge to the sparsity lecture.
- **Rush (Harvard NLP) (2018).** *The annotated transformer (web page)*. `https://nlp.seas.harvard.edu/annotated-transformer/`. The 2017 paper interleaved with a working PyTorch implementation, section by section. Secondary source.

Index of terms

$12Ld^2$ rule, 32
$12d^2$ rule, 31
$6ND$ rule, 41

absolute position, 13
AdamW, 38
analogy (vector arithmetic), 11
architecture, 44
arithmetic intensity, 41
aspect ratio, 32
attention, 15
attention matrix A, 18
attention sink, 23
attention weight, 15
autoregressive, 5

backpropagation, 38
bandwidth-bound, 37
base frequency, 14
batch, 38
batch normalization, 30
beam search, 35
BERT, 43
bias, 25
bidirectional attention, 21
block (layer), 6
byte-level BPE, 9
byte-pair encoding (BPE), 9

causal, 8
causal mask, 21
checked value, 45
circuit, 24
classical NLP pipeline, 32
compute-bound, 37
concatenation, 22
configuration, 40
configuration file, 45
context, 5
context extension, 14
contextual representation, 11
coreference, 15
cross-attention, 43
cross-entropy loss, 38
crossover $T = 6d$, 41

data-dependent routing, 15
dead unit, 27
decode, 37
decoder-only, 4
decoding (sampling) strategy, 35
depth, 6
dot product, 17

elementwise product $\odot$, 27
embedding, 6
embedding space, 11
embedding table, 10
encoder–decoder, 43
encoder-only, 43
end-of-text token, 9
epsilon ε, 30

final norm, 28
final vector h_T, 34
FlashAttention, 18
forward pass, 44

gain and bias γ, β, 29
gated linear unit (GLU), 26
GELU, 26
GiB, 37
gradient descent, 38
greedy decoding, 35
grouped-query attention (GQA), 22, 43

head, 22
head width d_k, 18
hidden unit, 25
hidden width d_{ff}, 25

identity path, 28
illustrative value, 45
in-context learning, 23
induction head, 23
input matrix X, 10
interpretability, 23

Jacobian, 28

key, 17
key–value memory (MLP reading), 25
key/value heads h_{kv}, 37
KV cache, 21, 37

large language model (LLM), 4
layer specialisation, 32
LayerNorm, 29
leading space, 9
learned position table, 13
learning-rate warm-up, 28
length independence, 31
local-window head, 23
logit, 34
lookup, 10
lower triangle, 19

mask M, 18
maximum context length $T_{\max}$, 13
memory-bandwidth bound, 41
MLP (feed-forward network), 25
model card, 44
multi-head attention (MHA), 22
multi-query attention (MQA), 43
multiply–add, 41

next-token prediction, 5
non-embedding parameters, 40
nonlinearity (activation function), 25
norm gains (per block), 31
normalisation (of probabilities), 34
normalization layer, 29
number of heads h, 22

one-hot vector, 10
output head, 34
output projection W_O, 22

parameter, 4
parameter count, 40
permutation invariance, 13
perplexity, 5
polysemy, 11
position encoding, 13
position table parameters, 40
position-wise, 7
post-norm, 28
pre-norm, 28
prefill, 37
prefix, 21
previous-token head, 23
probing, 32
projection matrices, 17

quadratic attention cost, 41
query, 17
query and key, 14

reader (query position), 19
relative position, 13
ReLU, 26
renormalisation, 35
residual connection, 7
residual stream, 7
RMSNorm, 29
root mean square (RMS), 29
rotary position embedding (RoPE), 14
rotation matrix, 14
row-wise softmax, 19

sampling, 5
scaled dot-product attention, 18
score, 17
score matrix, 18
self-attention, 15
self-supervised (next-token) training, 38
shape preservation, 31
shift invariance (of softmax), 19
sigmoid, 26
SiLU (Swish), 26
sinusoidal encoding, 13
soft lookup, 17
softmax, 6
softmax saturation, 18
stack, 32
sublayer, 6
subword, 9
SwiGLU, 26

T5, 43
target shift, 38
teacher forcing, 21
temperature, 35
tied parameters, 40
tiling, 41
token, 4
token ID, 9
tokenizer, 9
top-k sampling, 35
top-p (nucleus) sampling, 35
training tokens D, 38
transformer, 4
transformer block, 31

unembedding, 6
unembedding matrix W_U, 34

value, 17
vanishing or exploding gradients, 28
vocabulary, 5
vocabulary size V, 9

weight tying, 34
width (model dimension), 6
word2vec, 11